

Type|Racer

Documentation technique

Un jeu de vitesse de frappe embarqué dans Discord comme Activity.

Table des matières

1 Identification du projet	2
2 Le projet en chiffres	3
3 Description sommaire	4
3.1 Le produit	4
3.2 Le contexte Discord	4
3.3 Le vocabulaire minimal	4
4 Fonctionnalités	6
4.1 Le Menu	6
4.2 Solo : Practice	6
4.3 Solo : Apprendre	8
4.4 Multijoueur : entrer dans une Room	8
4.5 Multijoueur : le salon d'attente	9
4.6 Multijoueur : la course	9
4.7 Multijoueur : Floor is lava	10
4.8 Multijoueur : Spam	10
4.9 Multijoueur : le podium	11
4.10 Multijoueur : Play of the Game	12
4.11 Paramètres	12
4.12 Hors du jeu : ce que Discord montre	13
5 Réalisation du projet	14
5.1 Les trois axes d'une Race	14
5.2 Les trois Modes de jeu	14
5.3 La machine d'état d'une Race	15
5.4 Les Réglages de salon	16
5.5 Le multijoueur	16
5.6 Le cursus « Apprendre »	17
5.7 L'intégration Discord	17
5.8 Le mode debug dans l'iframe	18
5.9 La sécurité	19
5.10 Le pipeline d'assets et la Rich Presence	19
5.11 Le graphe de résultats, sans bibliothèque	22
5.12 La chaîne d'intégration et le déploiement	22
5.13 Méthode : le modèle de domaine fait autorité sur le nommage	24
5.14 Méthode : la parité TypeScript ↔ Rust prouvée par vecteurs	24
5.15 Méthode : les décisions d'architecture consignées	25
6 Documentation technique	26
6.1 Arborescence du projet	26
6.2 Architecture générale et frontière client/serveur	27
6.3 Le protocole WebSocket	28
6.4 Le contrat HTTP	36
6.5 Persistance	37
6.6 Sécurité et frontière de confiance	38
6.7 Tests et assurance qualité	39
7 Limites connues et suites	40
8 Annexes	41
8.1 Les dix-neuf décisions d'architecture	41
8.2 Les documents légaux	41
8.3 Où aller ensuite	42

1 Identification du projet

TypeRacerDiscord est un jeu de vitesse de frappe qui se joue **à l'intérieur de Discord**, comme Activity : une iframe lancée depuis un salon vocal, sans rien à installer et sans compte à créer. On y tape seul contre le chrono, ou à huit sur le même texte, avec trois règles de victoire différentes et un cursus de cent leçons pour apprendre à taper sans regarder ses doigts.

Le produit final est un binaire Linux unique, statique, qui sert à la fois l'interface et son API, publié en archive par la chaîne d'intégration à chaque version — plus les quelques assets déposés dans le portail développeur Discord.

Ce document décrit l'application telle qu'elle est à la version 0.0.4 : ce qu'elle fait pour un joueur, comment chacun de ses blocs a été construit, et comment le système tient debout une fois l'ensemble assemblé.

LA PILE Deux programmes, deux langages, une seule origine

FRONTEND

TypeScript, **sans cadriciel** — le DOM est manipulé directement. Build Vite. Un domaine pur (core/) qui ne touche ni au DOM ni au réseau, et une couche de rendu (ui/) qui ne décide de rien.

TEMPS RÉEL

Un WebSocket par joueur, 29 messages. Le serveur possède la graine, le texte et l'instant de départ ; le client ne possède que ce qu'il a tapé.

BACKEND

Rust, Axum, SQLite. Deux rôles dans un seul binaire : il sert le build du frontend **et** expose l'API. C'est ce qui permet de ne déclarer qu'une seule adresse à Discord.

LIVRABLE

Un binaire `x86_64-unknown-linux-musl` statique, produit par la chaîne d'intégration à chaque version, avec le build du frontend et une notice de déploiement générée.

La ligne qui traverse tout le document est celle-ci : **le même algorithme de score existe deux fois**, en TypeScript pour l'affichage en direct et en Rust pour le verdict. Aucun générateur ne relie les deux. C'est un choix, il a un prix, et le prix se paie en section 5.14.

2 Le projet en chiffres

Un instantané, mesuré sur le dépôt à la version 0.0.4. Ces nombres ne sont pas un palmarès : ils donnent l'ordre de grandeur qu'il faut avoir en tête pour lire les sections suivantes — notamment le rapport entre le code de production et ce qui l'atteste.

11 368

lignes TypeScript
frontend/src, tests compris

7 960

lignes Rust
backend/src, tests compris

2 312

lignes de feuille de style
une seule, et c'est voulu

382 + 159

tests automatisés
vitest · cargo test

29

messages du protocole
20 montants, 9 descendants

19

décisions d'architecture
consignées, une par fichier

100

leçons du cursus
une donnée, pas du code

6

migrations de base
dont deux correctives

3

Modes de jeu multijoueur
des axes, jamais cumulables

2

langages tenus à parité
prouvée par vecteurs partagés

121

issues fermées
une seule encore ouverte

1

binaire à déployer
statique, il emporte tout

Figure 1. – Le projet mesuré, version 0.0.4. Pas de filets ni de colonnes alignées : ces douze nombres ne se comparent pas entre eux, et les ranger en tableau laisserait croire le contraire.

Deux chiffres méritent d'être lus ensemble. **Deux langages tenus à parité** signifie que le même algorithme de score existe en TypeScript et en Rust, et que des vecteurs de test communs interdisent aux deux versions de diverger — c'est le sujet de la section 5.14. **29 messages** est la taille du contrat entre le navigateur et le serveur pendant une course ; c'est aussi ce qui dérivera en premier si le protocole bouge, et ce que section 6.3 liste un par un.

3 Description sommaire

3.1 Le produit

Un joueur ouvre Discord, entre dans un salon vocal, clique sur la fusée et lance TypeRacer. L'application s'affiche dans une iframe, à l'intérieur du client Discord — pas dans un onglet, pas dans une fenêtre à côté. Elle reconnaît le joueur toute seule : son nom et son avatar viennent de la session Discord déjà ouverte, il n'y a rien à saisir.

De là, deux chemins. **Solo** : un texte apparaît, le chrono part à la première frappe, et l'écran de résultats affiche vitesse, précision et une courbe seconde par seconde. **Multijoueur** : les joueurs du même salon se retrouvent automatiquement dans un salon d'attente, l'hôte règle la partie, et tout le monde tape le même texte pendant que des voitures avancent sur une piste. Un troisième chemin, **Apprendre**, est un cursus de cent leçons qui ne se joue pas contre les autres mais contre ses propres doigts.

Pendant tout ce temps, les autres membres du salon vocal voient dans Discord ce que le joueur est en train de faire — « En course », « S'entraîne », avec le visuel du mode choisi. C'est la Rich Presence, et elle fonctionne même pour ceux qui n'ont pas ouvert l'Activity.

3.2 Le contexte Discord

Une Activity n'est pas un site web hébergé quelque part que Discord irait ouvrir. C'est une page servie **à travers Discord**, dans une iframe verrouillée, sous une politique de sécurité de contenu qui refuse toute requête réseau ne passant pas par un préfixe imposé. Cette contrainte-là a façonné plus de code que n'importe quel choix d'architecture — elle revient à la section 5.7.

Trois conséquences structurent tout le reste :

- **L'origine est unique.** Discord ne redirige qu'une seule adresse vers le projet. Le serveur Rust sert donc à la fois le build du frontend et l'API.
- **Il n'y a pas d'URL.** L'adresse de l'iframe est figée par Discord ; la navigation se fait entièrement par boutons, écran par écran.
- **Il n'y a pas de console.** Un développeur qui déboguerait dans le client Discord ne voit rien du tout — d'où un bandeau d'erreurs affiché dans le jeu lui-même (section 5.8).

3.3 Le vocabulaire minimal

Le projet tient un glossaire de domaine complet — une quarantaine de termes, avec pour chacun les mots qu'il est interdit d'employer à sa place — dans `CONTEXT.md`, à la racine du dépôt. Ce document n'en reprend pas la lettre. Voici les douze termes sans lesquels les sections suivantes ne se lisent pas ; les autres sont introduits là où ils servent.

TERME	CE QU'IL DÉSIGNE
Run	Une tentative de frappe, du premier caractère à la fin. L'unité à laquelle tout le reste s'accroche. Deux espèces : Practice et Race.
Practice	Un Run solo en saisie libre : le retour arrière est permis, une faute peut rester non corrigée, on peut taper au-delà du mot.
Race	Un Run compétitif dans une Room, sur la même saisie libre — mais qui ne se termine qu'une fois le texte entier tapé exactement .
Room	La session multijoueur qui réunit les joueurs sur un même texte. Identifiée par une clé unique prenant deux formes : un salon vocal, ou un Code de partie.

TERME	CE QU'IL DÉSIGNE
Mode	La règle solo qui décide du texte présenté et de la fin du Run : Time, Words, Quotes ou Zen. Exactement un par Run — et aucun en Race.
Mode de jeu	La règle qui décide comment une Race se gagne : Normal, Floor is lava ou Spam. Un axe à part entière, jamais cumulable.
Source de texte	D'où vient le texte d'une Race — une citation, ou des mots générés. Elle décide du texte, jamais de la mesure.
Réglage de salon	Une option de Room posée par l'hôte hors course et imposée à tout le monde : durée du décompte, taille max, Difficulté, Mode de jeu...
Preference	Ce qu'un joueur veut voir sur sa machine à lui : police, palette, pseudo affiché. Ne quitte jamais l'appareil et n'influence aucun score.
Keystroke log	La chronologie brute des frappes d'un Run : quoi, et à quel instant. La matière première dont tout le reste est dérivé.
Scoreboard autoritaire	Les chiffres qui font foi, recalculés par le serveur Rust depuis le Keystroke log à la fin d'un Run. En multijoueur, c'est aussi l'anti-triche.
Gap (écart)	De combien de secondes un joueur a fini derrière le vainqueur. C'est le chiffre qu'on dit à voix haute à l'arrivée, pas la vitesse absolue.

Tableau 1. – Les douze termes de travail, en tableau zébré : on les lit une fois de haut en bas, et la rayure garde l'œil sur la bonne ligne quand la définition tient sur trois. Le glossaire complet, avec ses interdits de vocabulaire, reste dans `CONTEXT.md`.

EN CLAIR Le « recompute autoritaire »

Le navigateur du joueur affiche un compteur de vitesse pendant qu'il tape, mais ce chiffre-là n'est **jamais** celui qu'on enregistre. À la fin, le navigateur envoie au serveur la liste brute de ses frappes — la touche et l'instant, rien d'autre. Le serveur **rejoue** cette liste contre le texte qu'il possède, et recalcule tout lui-même. C'est un peu comme rendre sa copie d'examen : ce n'est pas la note que l'élève s'est donnée qui compte, c'est celle du correcteur. Le joueur peut mentir sur sa vitesse, il ne peut pas mentir sur ses frappes sans réécrire une partie entière cohérente.

4 Fonctionnalités

Cette section décrit le produit **à l'œil du joueur** : ce qu'il voit, dans l'ordre où il le rencontre. Aucun nom de fichier, aucun détail d'implémentation — ceux-là viennent à la section 5 et à la section 6.

4.1 Le Menu

Le point d'arrivée. Quatre portes : Solo, Multijoueur, Paramètres, Quitter. « Quitter » ferme réellement l'Activity dans Discord, et disparaît quand le jeu tourne hors Discord.

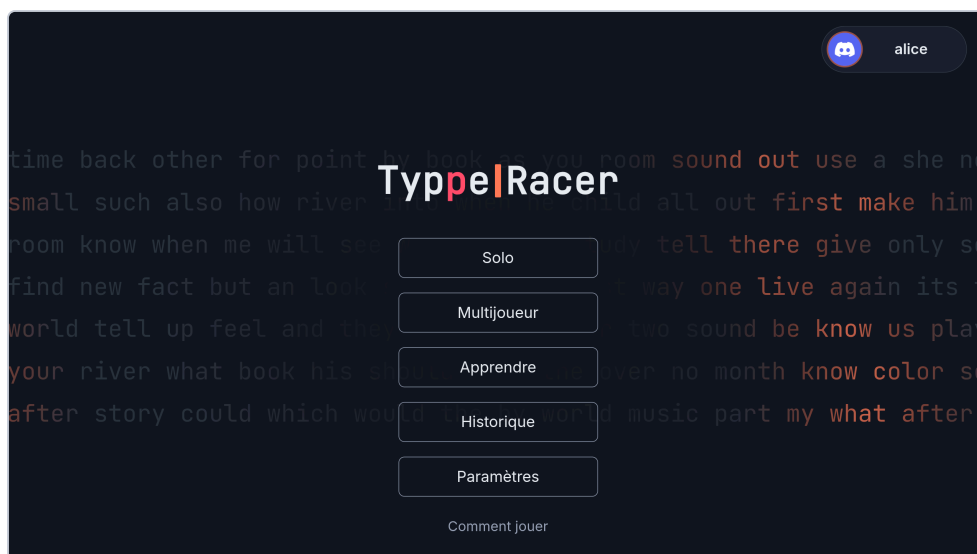


Figure 1. – Le Menu, écran d'arrivée. Les quatre entrées, le wordmark, et l'accès au guide « Comment jouer » — affiché d'office la toute première fois.

4.2 Solo : Practice

Une barre de configuration au-dessus de la zone de frappe : le Mode (Time / Words / Quotes / Zen), sa valeur, puis les modificateurs de texte cumulables — ponctuation, chiffres. Deux Modes particuliers escamotent ces contrôles parce qu'ils ne s'y appliquent pas : Quotes (la longueur appartient à la citation) et Zen (il n'y a pas de texte cible du tout).

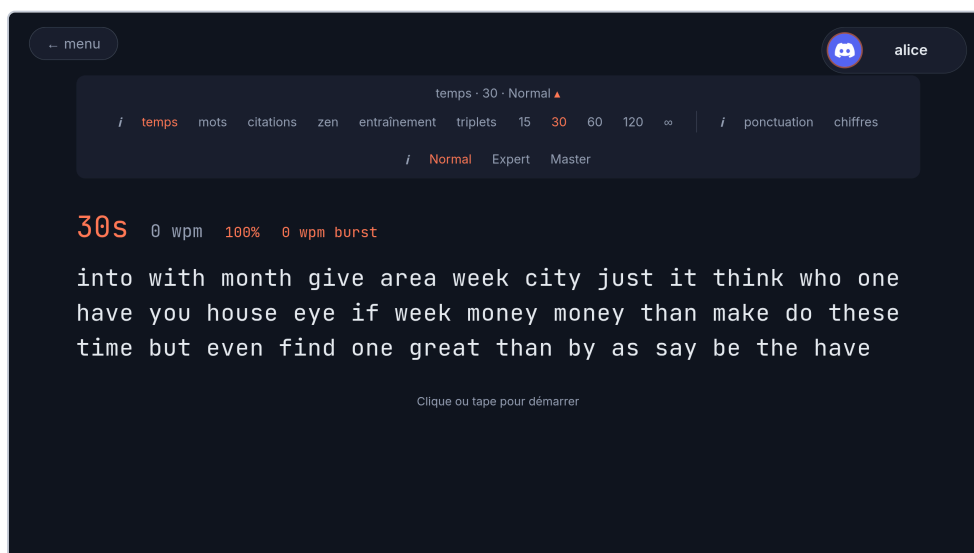


Figure 2. – La barre de configuration solo dépliée. Chaque contrôle n'apparaît que lorsqu'il a un sens : choisir « citation » fait disparaître la longueur et les modificateurs.

Le chrono ne part pas au clic mais à la **première frappe** : rien n'est masqué, personne n'attend, il n'y a pas de décompte. Le texte défile par fenêtre de trois lignes, le mot en cours restant toujours sur celle du milieu.

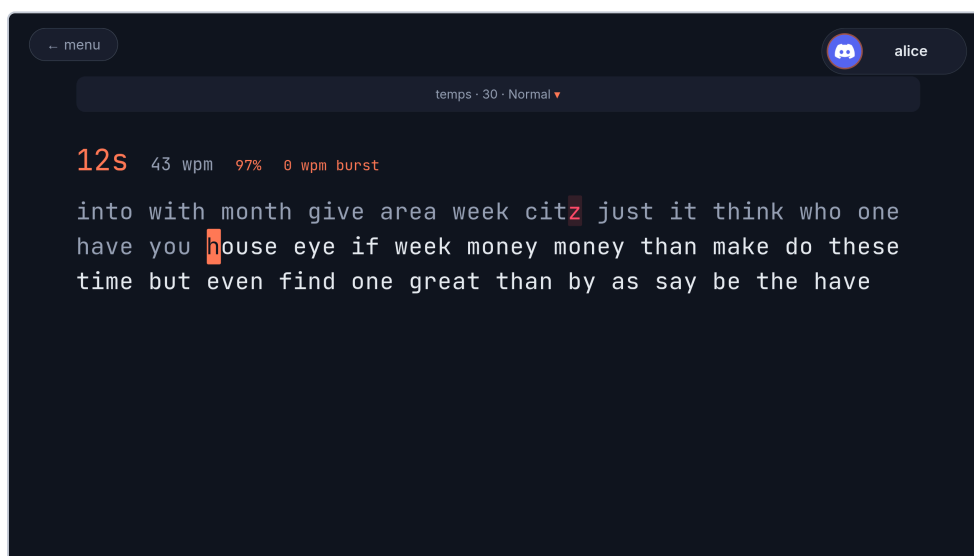


Figure 3. – Un Practice en cours. Le mot actif reste sur la ligne du milieu ; les caractères déjà tapés changent de couleur, une faute passe en rouge.

À la fin, l'écran de résultats affiche le scoreboard recalculé par le serveur — vitesse nette, vitesse brute, précision, décompte des caractères — et la courbe seconde par seconde. Un bouton rejoue le Run à sa vitesse réelle, fautes comprises.



Figure 4. – L'écran de résultats. Le graphe seconde par seconde est un SVG construit à la main : la dépendance à une bibliothèque de graphiques a été retirée du projet (section 5.11).

4.3 Solo : Apprendre

Un cursus de cent leçons, débloquentes une à une. Chacune enseigne un point précis — la position des mains, la rangée de repos, les majuscules, la ponctuation, les chiffres, puis les vrais mots — et se termine par un exercice. Le seul critère pour passer à la suivante est la **précision**, jamais la vitesse, et la barre monte par paliers au fil du cursus.



Figure 5. – La liste des leçons. Trois états visibles : verrouillée, disponible, terminée. La progression suit le joueur, pas l'appareil.

4.4 Multijoueur : entrer dans une Room

Trois portes, présentées ensemble sur le même écran :

Le salon — rejoindre la Room du salon vocal courant. Elle est créée à la volée si elle n'existe pas — la clé vient de Discord, elle ne peut pas être mal tapée.

Créer — ouvrir une Room neuve et recevoir un Code de partie de cinq caractères, lisible à voix haute.

Rejoindre par code — saisir un code. Celui-ci ne crée jamais rien : un code inconnu répond « introuvable » et laisse corriger sur place.

Le code est ce qui permet à des joueurs de **serveurs Discord différents** de courir ensemble.

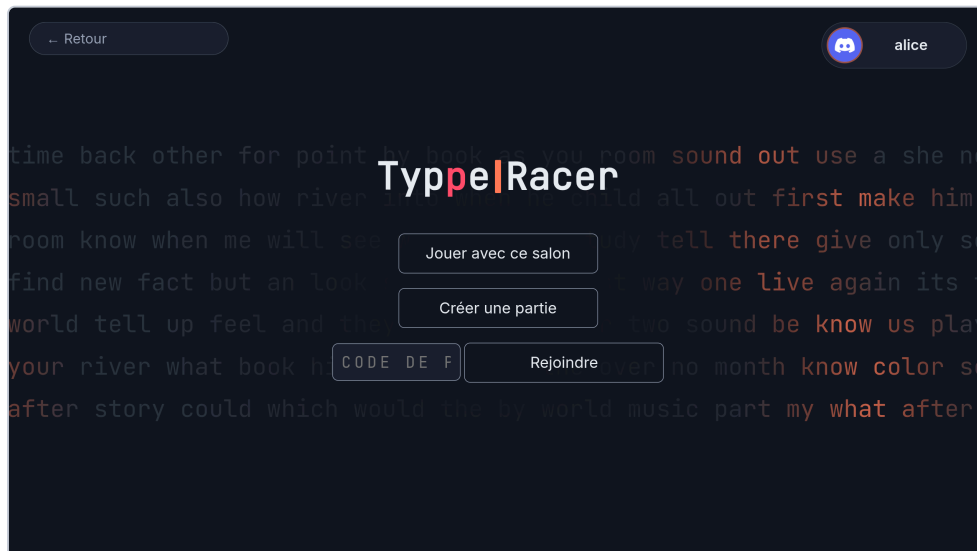


Figure 6. – Les trois portes d’entrée d’une Room, avec le champ de saisie du code au même endroit : un code refusé ramène le joueur là où il peut le corriger.

4.5 Multijoueur : le salon d’attente

Trois colonnes. À gauche les présents, avec avatar, pseudo et couronne pour l’hôte. Au centre les réglages de la partie. À droite, dans un cadre, le visuel du Mode de jeu choisi et ce qu’il implique.

Ce que l’hôte règle : le Mode de jeu, la Source du texte et sa longueur, la durée du décompte, la taille maximale de la Room, la Difficulté, le ready-check — et, selon le Mode de jeu, l’intervalle d’élimination ou le mot et les seuils. Tout le monde voit ces réglages : ils s’imposent à tous, pas seulement à celui qui les choisit.

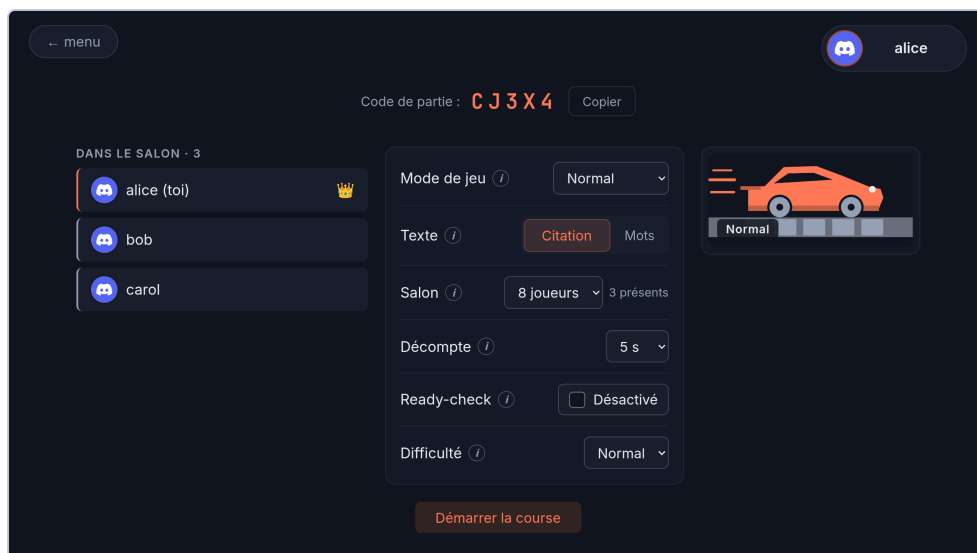


Figure 7. – Le salon en trois colonnes. Le Code de partie est masqué par défaut et se révèle d’un clic — on ne diffuse pas par accident l’entrée de sa partie.

4.6 Multijoueur : la course

Une ligne par joueur : l’avatar en tête de progression, le pseudo, la vitesse en direct à hauteur de la ligne d’arrivée. Le décompte dure ce que l’hôte a réglé, texte entier visible pendant l’attente — il n’y a rien à masquer, tout le monde part au même instant.



Figure 8. – La piste pendant une course classique. La position de chaque voiture suit le nombre de caractères corrects, pas le temps.

4.7 Multijoueur : Floor is lava

À intervalle régulier, le joueur le moins avancé brûle. La course s'arrête à l'instant où il ne reste qu'un vivant : le survivant gagne **sans avoir terminé le texte**. C'est la seule course qui se finit sans que personne ne franchisse de ligne d'arrivée — le mode impose d'ailleurs un texte trop long pour être achevé. On ne gagne pas en tapant vite, on gagne en n'étant jamais dernier.

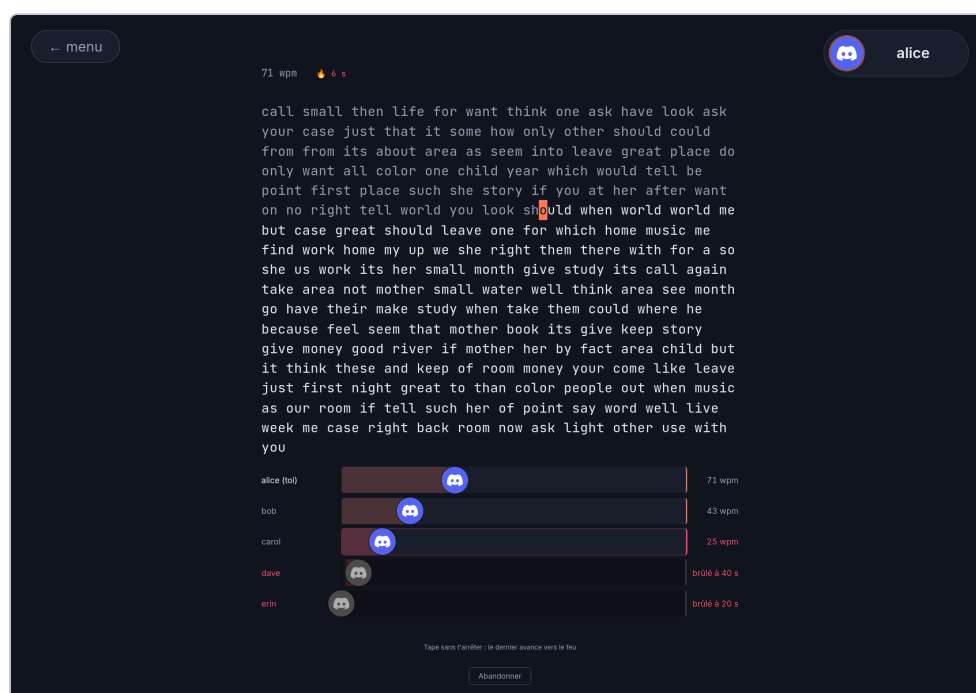


Figure 9. – Floor is lava en action. Ce qui compte n'est pas la position absolue mais la position **relative** : le dernier au moment du tic disparaît.

4.8 Multijoueur : Spam

Un seul mot, répété indéfiniment. L'hôte choisit le mot — dans la liste existante ou en le tapant lui-même — un seuil de répétitions, et un plafond de temps. La course s'arrête dès que l'un des deux tombe : soit quelqu'un verrouille le seuil, soit le temps expire et c'est le plus grand nombre de répétitions correctes qui l'emporte.



Figure 10. – Le mode Spam et son compteur de répétitions, qui remplace la vitesse à la ligne d'arrivée : c'est la grandeur qui décide de la victoire.

4.9 Multijoueur : le podium

Trois marches, les autres joueurs visibles à côté, et le **Gap** en gros caractères — l'écart au vainqueur en secondes. Cliquer sur un joueur déplie sa courbe seconde par seconde, sans aucune requête réseau : tout est arrivé avec le message de fin de course.

Les deux Modes de jeu n'ont pas de Gap, puisque personne ne franchit de ligne. Chacun affiche à sa place ce qui décide vraiment de son classement : le temps de survie pour Floor is lava, le nombre de répétitions pour Spam.



Figure 11. – Le podium, Gap en tête d'affiche. Les abandons et les échecs figurent derrière tous les finisseurs, jamais mêlés à eux.

4.10 Multijoueur : Play of the Game

Quand deux joueurs ont fini à moins de deux secondes l'un de l'autre — où qu'ils soient au classement — leurs dernières secondes sont rejouées côte à côte, au ralenti, sur une **horloge partagée**. C'est cette horloge unique qui en fait un duel plutôt que deux relectures posées l'une à côté de l'autre. Sans photo-finish, il n'y a pas de Play of the Game du tout : le bouton n'apparaît pas.

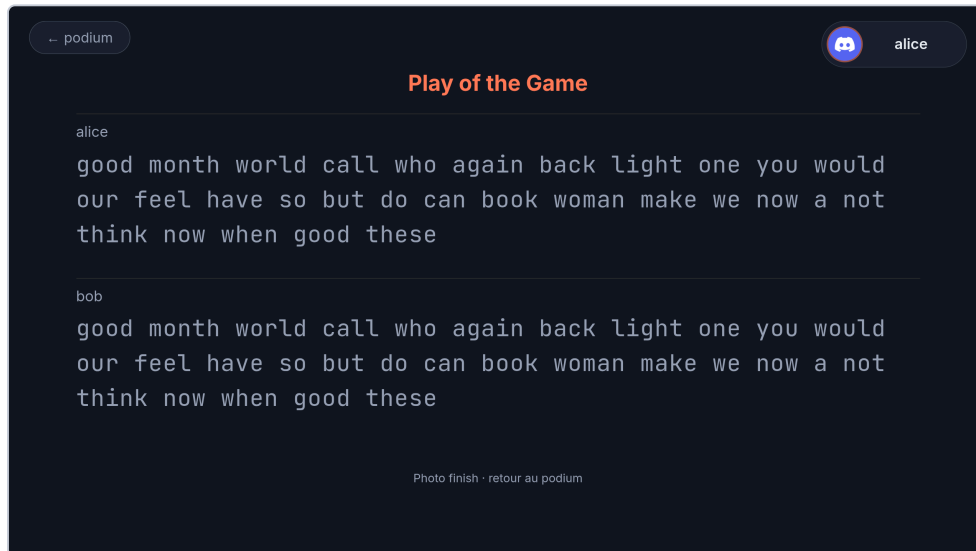


Figure 12. – Play of the Game : deux Keystroke logs rejoués sur la même horloge, sur les dernières secondes avant la première des deux arrivées.

4.11 Paramètres

Une ligne par réglage : libellé et explication à gauche, contrôle à droite, groupés en sections — Apparence, Solo, Son, et une zone à risque pour exporter, importer ou effacer ses données. Le choix de la police de frappe s'affiche directement dans la police concernée : l'aperçu **est** le contrôle.

Ces réglages ne quittent jamais l'appareil. Deux joueurs de la même course peuvent voir des polices et des couleurs différentes tout en tapant exactement le même texte, et changer un réglage n'invalide jamais un record.

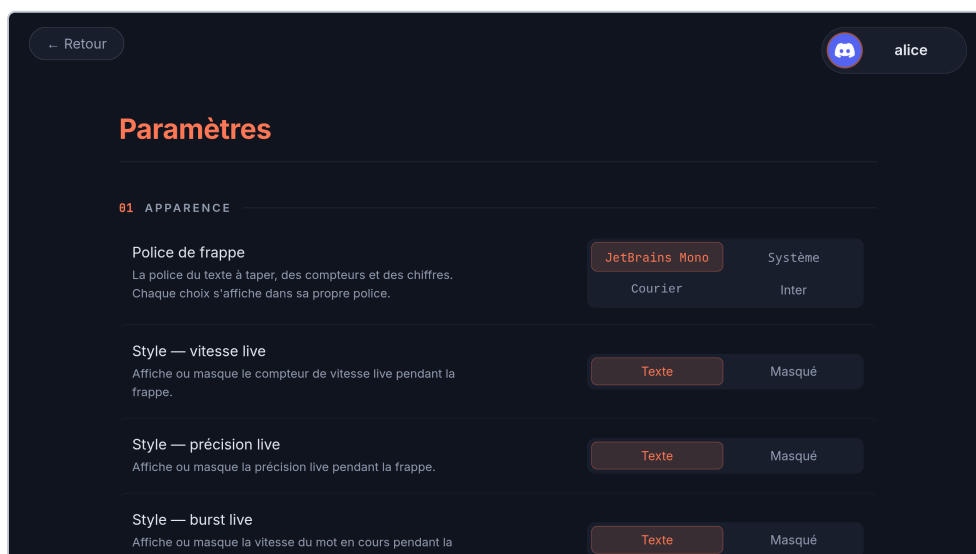


Figure 13. – L'écran Paramètres. Chaque type de contrôle — segmenté, curseur, bascule, nombre — est né avec le premier réglage qui en avait besoin.

4.12 Hors du jeu : ce que Discord montre

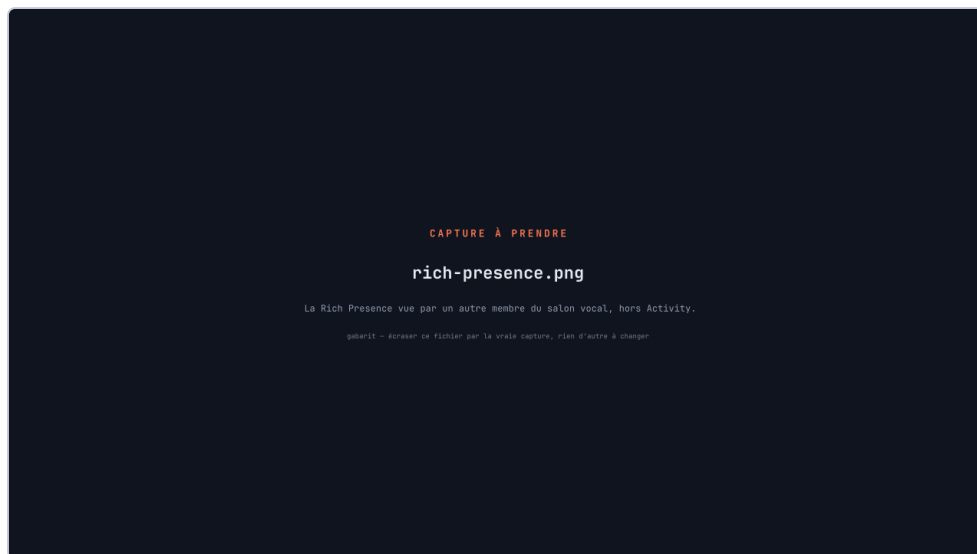


Figure 14. – La Rich Presence telle qu'un autre membre du salon la voit, sans avoir ouvert l'Activity : l'état courant, le visuel du Mode de jeu, le compte des présents et un chrono. C'est la seule preuve visible que le câblage fonctionne de bout en bout.

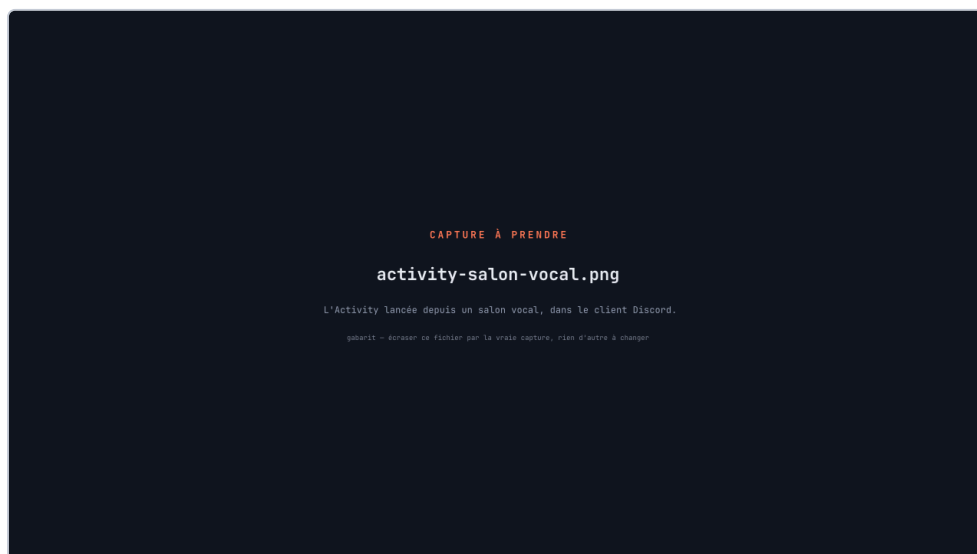


Figure 15. – L'Activity lancée depuis un salon vocal, dans le client Discord. L'iframe occupe la zone de conversation ; le jeu s'échelonne à la fenêtre plutôt que de faire défiler.

5 Réalisation du projet

Cette section change de point de vue : on ne regarde plus l'écran, on regarde le chantier. Un paragraphe descriptif par bloc construit, dans l'ordre où ils s'empilent. Les encadrés « récit » n'interrompent le catalogue que pour les problèmes qui ont **réellement** résisté — un chantier qui s'est bien passé n'a pas d'histoire à raconter, et un récit posé partout ne raconterait plus rien.

Les trois derniers chantiers ne produisent aucune fonctionnalité : ce sont des chantiers de **méthode**, et ils expliquent pourquoi les précédents ont pu être menés sans se marcher dessus.

5.1 Les trois axes d'une Race

Le concept le plus facile à confondre du domaine, et celui qu'il a fallu séparer en premier. Trois réglages agissent sur une même course, et **aucun** n'est une variante des deux autres.

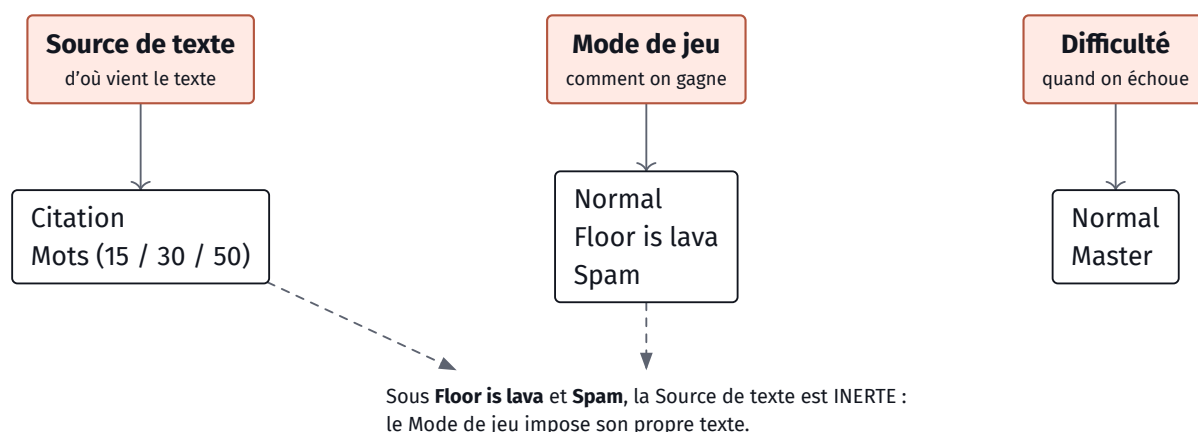


Figure 16. – Les trois axes d'une Race. La Source décide du texte et jamais de la mesure ; le Mode de jeu décide de la victoire ; la Difficulté est une condition d'échec **individuelle**, évaluée sur le seul journal de frappe du joueur concerné et jamais contre les autres.

Une Race n'a délibérément **pas** de Mode : sa règle de fin est toujours « le texte entier, exactement ». Un Mode Time en course impliquerait des voitures sans ligne d'arrivée commune, et Zen n'a pas de fin du tout. Ce que l'hôte choisit, c'est la provenance du texte — le recalcul autoritaire, lui, mesure toujours la même chose.

5.2 Les trois Modes de jeu

Normal est le comportement d'origine : le premier à taper tout le texte, exactement. Les deux autres ont été ajoutés comme des **axes**, pas comme des options, précisément pour qu'ils ne se cumulent jamais entre eux.

Floor is lava remplace la victoire par la survie. Un métronome serveur bat à l'intervalle réglé ; à chaque battement, le moins avancé brûle. Le classement est l'ordre des morts, inversé. Deux conséquences ont dû être tirées jusqu'au bout : le mode impose son propre texte, assez long pour que personne ne l'achève — un mode qui se gagne en survivant ne doit pas offrir de porte de sortie par l'arrivée — et un joueur brûlé porte tout de même un **vrai score partiel**, recalculé sur ce qu'il a eu le temps de taper. Ce score ne le classe jamais ; il sert à l'afficher et à choisir le duel d'après-course.

Spam remplace le texte par un mot unique, diffusé sans fin. Deux réglages décident de l'arrêt — un seuil de répétitions et un plafond de temps — et le premier des deux qui tombe arrête tout le monde. Le

compte de répétitions annoncé par un client est déclaratif : il peut **arrêter** la course, il ne peut pas la gagner. Le classement vient du recompte serveur, qui rejoue chaque journal de frappe contre sa propre copie du mot.

RÉCIT Floor is lava éliminait pendant le décompte

Problème Le métronome démarrait avec la course, pas avec la fin du décompte. Pendant les cinq secondes d'attente, personne n'a encore tapé le moindre caractère : tout le monde est donc à égalité au dernier rang. Le premier battement éliminait alors **tous** les joueurs d'un coup, et la Room se figeait dans un état sans vivant et sans vainqueur.

Décision Faire partir le métronome au signal de départ partagé et non à la création de la course, et traiter l'égalité générale comme un cas nommé plutôt que comme un tri qui « tombe bien » la plupart du temps.

Résultat Le mode a servi d'avertissement pour les suivants : toute règle comparative doit dire ce qu'elle fait quand **tous** les joueurs sont à égalité, y compris à zéro. Le cas dégénéré n'est pas un cas rare, c'est l'état initial de chaque course (issue 159).

EN CLAIR Un texte « généré à partir d'une graine »

Le texte d'une course n'est pas tiré au hasard puis envoyé à tout le monde : on tire un **nombre**, la graine, et le texte se déduit de ce nombre par un calcul sans surprise. Même graine, même texte, toujours, sur n'importe quelle machine. C'est ce qui permet au serveur Rust et au navigateur de fabriquer le même texte chacun de son côté sans se le transmettre — et c'est aussi ce qui rend les tests possibles : une graine fixée donne un texte connu d'avance.

5.3 La machine d'état d'une Race

Chaque course traverse les mêmes phases, et chaque joueur en sort par l'une de cinq portes. Ces cinq sorties ne sont pas des nuances de la même chose : elles se distinguent par **qui décide**.

Une précision de vocabulaire, parce que le glossaire du projet compte différemment. CONTEXT.md réserve le mot « état terminal » aux quatre façons de sortir d'une course **sans l'avoir finie** — Abandon, Échec, Brûlé, Devancé — et traite « fini » à part, comme la sortie normale. Le code, lui, ne fait pas cette distinction : une fois posé, *finished* ne peut pas plus être écrasé que les quatre autres. Le diagramme ci-dessous compte donc les cinq, parce que c'est la propriété qu'il illustre — l'irréversibilité, pas la réussite.

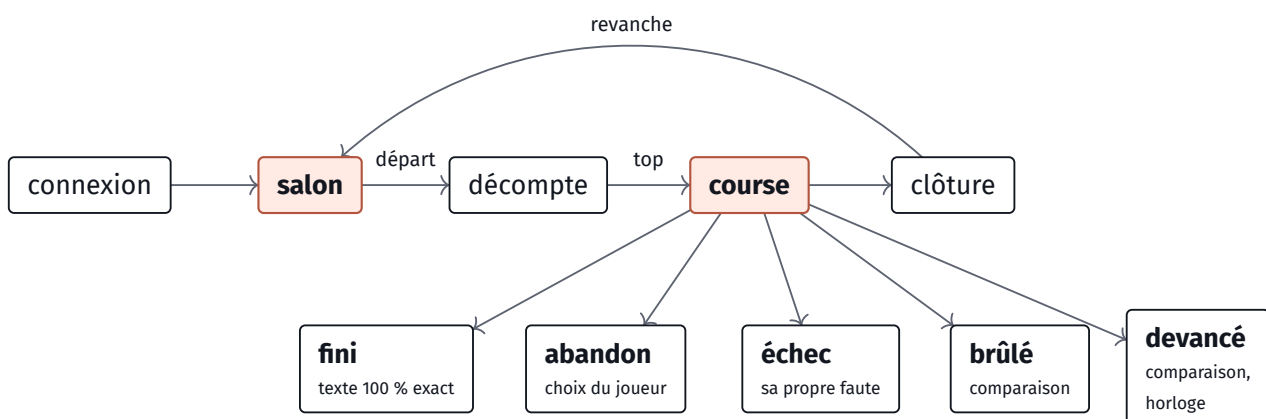


Figure 17. – Les phases d'une Race et les cinq états terminaux d'un partant. **Fini** est la seule sortie par la ligne d'arrivée. **Abandon** vient du joueur lui-même — et une déconnexion produit exactement le même enregistrement. **Échec** vient de sa propre faute sous Difficulté Master. **Brûlé** et **devancé** ne viennent ni d'un choix ni d'une faute : ils viennent d'être comparé aux autres.

Un état terminal ne se reprend pas : une fois posé, il ne peut plus être écrasé par un message arrivé en retard. Un abandon garde par ailleurs la position qu'avait le joueur — sa voiture s'arrête là où il s'est arrêté, elle ne retombe pas à zéro.

5.4 Les Réglages de salon

Six réglages généraux — Source de texte, taille maximale, durée du décompte, ready-check, Difficulté, Mode de jeu — plus quatre propres aux Modes de jeu : l'intervalle d'élimination de Floor is lava, et le mot, le seuil et le plafond de temps de Spam. Ils partagent tous la même frontière : posés par l'hôte seul, acceptés hors course seulement, et rediffusés à tout le salon dès qu'ils sont acceptés. Un réglage refusé est ignoré en silence côté serveur — le client n'est jamais en position d'imposer quoi que ce soit.

Le point délicat n'était pas la liste mais la **bascule**. Passer d'un Mode de jeu à un autre régénère le texte, puisque chaque mode impose le sien. Or les réglages préparés pour un mode qu'on quitte doivent survivre : celui qui règle Spam, passe voir Floor is lava, puis revient, doit retrouver son mot et ses seuils tels qu'il les avait laissés. Les réglages vivent donc à plat sur la Room plutôt que dans la variante du Mode de jeu — un écart assumé par rapport à la décision d'architecture qui les décrivait, et documenté comme tel dans le code.

5.5 Le multijoueur

Une Room est identifiée par une clé unique qui prend deux formes : un salon vocal Discord, ou un Code de partie de cinq caractères. Une seule table, deux formes — et trois portes d'entrée aux droits différents, décrites à la section 6.3. L'alphabet du code exclut les caractères qui se confondent à l'oral et à l'œil : ni 0/0, ni 1/I/L. Un code n'est jamais réservé ni persisté ; il vit tant que sa Room vit, et meurt avec elle.

Le premier arrivé est l'hôte, et le rôle passe au suivant s'il part. La Room plafonne à huit présents. Les partants sont **figés au départ** : quelqu'un qui rejoint pendant une course occupe une place mais n'est pas dans cette course-là.

À l'arrivée, le message de fin porte le classement complet de tous les partants, avec leur courbe seconde par seconde — pas seulement l'ordre. C'est ce qui permet au podium d'afficher le Gap et de déplier le graphe de n'importe quel joueur sans un seul aller-retour réseau. Ce choix a une raison précise : la composition d'une course vit en mémoire et meurt avec la Room, donc le serveur ne pourrait pas vérifier après coup qu'un demandeur en faisait partie. Sur le WebSocket, la question ne se pose pas — l'autorisation **est** la connexion.

RÉCIT Le verrou des Rooms et l'aller-retour réseau

- Problème** Une Room dont la Source est « citation » doit aller chercher son texte sur une API externe. Mais l'état des Rooms est protégé par un verrou classique, celui qui ne peut pas être tenu à travers une attente réseau : le tenir pendant la requête bloquerait toutes les autres Rooms du serveur pendant la latence de l'API, et le compilateur Rust refuse d'ailleurs la construction.
- Décision** Découper en trois temps : lire la Source sous verrou, **relâcher**, chercher le texte sans verrou, reposer le résultat sous verrou et rediffuser l'état. Et faire naître toute Room avec un texte de mots déjà en place, même quand sa Source est « citation ».
- Résultat** Une Room est jouable immédiatement, sans réseau. Le salon voit l'ancien texte puis le nouveau. Une course lancée entre-temps annule la pose du texte en vol, devenu périmé. Le repli après échec de l'API n'est pas un état à part — la Room bascule **réellement** sur des mots générés, et c'est ce qu'elle annonce.

EN CLAIR Pourquoi un verrou ne traverse pas une attente

Un verrou sert à garantir qu'une seule partie du programme touche une donnée à la fois. Le tenir, c'est faire attendre tous les autres. Tant qu'on ne fait que quelques calculs, l'attente se compte en microsecondes. Mais si on garde le verrou pendant qu'on interroge un serveur à l'autre bout d'Internet, on fait attendre tout le monde pendant des centaines de millisecondes — pour une requête qui ne concernait qu'une seule Room. D'où le découpage en trois temps : on ne tient jamais la porte fermée pendant qu'on va faire les courses.

5.6 Le cursus « Apprendre »

Cent leçons, du placement des mains à la fluidité, en passant par les rangées, les majuscules, la ponctuation et les chiffres. Le contenu est une **donnée** — un fichier de description séparé du moteur — précisément pour qu'ajouter ou réécrire une leçon ne demande pas de toucher au code.

Deux décisions structurent le cursus. La première : le seul critère de déverrouillage est la précision, jamais la vitesse, avec un barème par tranches qui se durcit au fil des leçons et se modifie en un seul endroit. La seconde : un exercice de leçon **n'est pas un Run**. Il n'entre pas dans l'historique, ne produit jamais de record, et n'est jamais soumis au serveur pour recalcul. La progression, elle, suit le joueur et non l'appareil — le serveur en conserve toujours le maximum atteint, jamais la dernière valeur reçue.

5.7 L'intégration Discord

L'identité du joueur ne vient d'aucun formulaire : elle vient de la session Discord déjà ouverte, par un enchaînement en quatre temps entre l'iframe, le client Discord et le serveur.

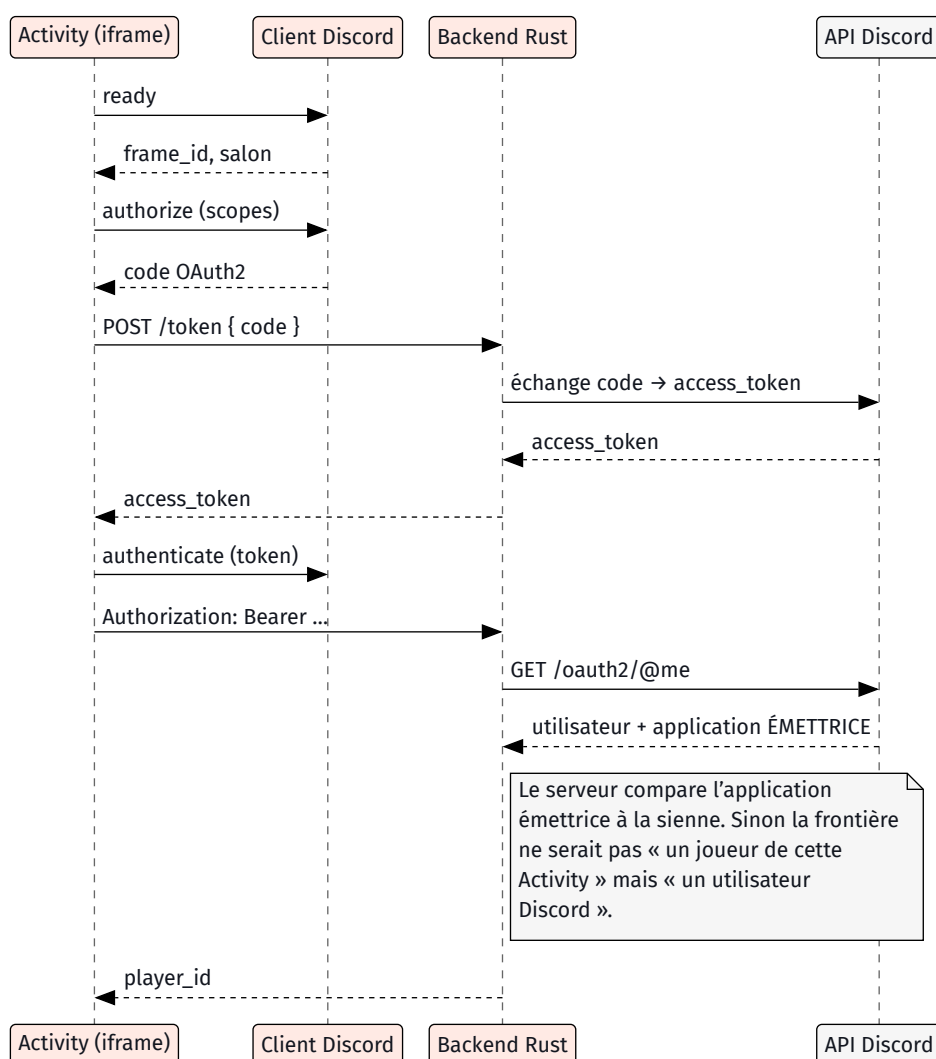


Figure 18. – Le handshake d'identité. Le secret client ne quitte jamais le serveur : c'est lui, et non l'iframe, qui échange le code contre un jeton. La dernière vérification est celle qui compte — un jeton d'accès Discord est valable sur l'endpoint d'identité **quelle que soit l'application qui l'a obtenu** (issue 150).

EN CLAIR OAuth2, ou « prouver son identité sans donner son mot de passe »

Le jeu ne connaît jamais le mot de passe Discord de personne. Discord remet à l'application un **jeton** temporaire, qui dit « ce porteur est bien tel utilisateur, et il a accepté de partager son nom ». Le jeu montre ce jeton au serveur, qui va demander à Discord à qui il appartient. Le piège que le projet a dû traiter : un jeton obtenu par **une autre** application Discord répond aussi à cette question. Il faut donc demander en plus qui l'a émis, sinon n'importe quel utilisateur de Discord pourrait se présenter comme joueur.

Le reste de l'intégration tient dans deux contraintes. La première est la politique de sécurité de l'iframe, sujet du récit ci-dessous. La seconde est qu'une Activity non publiée reste **invisible** tant qu'un testeur n'a pas accepté son invitation par courriel — un piège de portail qui n'a rien à voir avec le code, et qui a coûté une session entière avant d'être identifié.

RÉCIT La politique de sécurité de l'iframe Discord

Problème Dans l'iframe d'une Activity, **toute** requête réseau qui ne passe pas par un préfixe imposé est refusée. Le document et les scripts se chargent normalement : l'interface s'affiche, parfaitement intacte, et rien ne fonctionne. Aucune erreur visible, aucun message — et la console du navigateur n'existe pas dans le client Discord.

Décision Une seule fonction décide du préfixe, en observant si l'application tourne dans une iframe Discord ou non, et les quatre points d'accès réseau du projet passent tous par elle. Aucun appel direct nulle part.

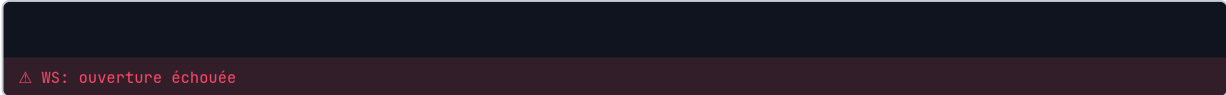
Résultat Le proxy Discord retire le préfixe **avant** d'appliquer sa redirection : le serveur et le développement local n'ont donc rien à savoir de tout ça. Ce symptôme — « l'interface va bien, le réseau est mort » — est devenu le premier réflexe de diagnostic du projet, et il a directement motivé le bandeau d'erreurs de la section suivante.

EN CLAIR La politique de sécurité de contenu d'une iframe

Une page web peut déclarer une liste de ce que le navigateur a le droit de charger et de contacter. Discord en impose une aux Activities, et elle est très serrée : rien ne sort sans passer par un chemin balisé qu'elle contrôle. L'intention est saine — une Activity malveillante ne peut pas discrètement envoyer les données de ses joueurs ailleurs. L'effet secondaire est cruel : une requête refusée ne casse pas la page, elle échoue silencieusement.

5.8 Le mode debug dans l'iframe

Corollaire direct du récit précédent : sans console, une erreur JavaScript dans Discord est totalement invisible. Le jeu attrape donc lui-même les erreurs non rattrapées et les promesses rejetées, et les affiche dans un bandeau rouge en bas de l'écran, refermable d'un clic. C'est une dizaine de lignes, et c'est ce qui a rendu le débogage dans Discord possible.



△ WS: ouverture échouée

Figure 19. – Le bandeau d'erreurs, recadré sur la bande elle-même — le reste de l'écran est celui de la capture précédente, et le montrer deux fois n'apprendrait rien. Ici le backend est arrêté : la promesse d'ouverture du WebSocket part en rejet, personne ne la rattrape, et c'est cette bande qui le dit. Elle n'existe que parce que la console n'existe pas — dans le client Discord, c'est le seul canal de diagnostic disponible.

Pour une vraie console, il reste un chemin : ouvrir Discord **au navigateur** et lancer l'activité de là. Les outils de développement sont alors ceux du navigateur, sur l'iframe.

5.9 La sécurité

La passe de sécurité a porté sur cinq fronts distincts, tous menés après que l'application ait été jouable — et c'est précisément ce qui a permis de les traiter comme des frontières plutôt que comme des rustines.

La frontière d'identité — le jeton doit avoir été émis pour **cette** application, pas seulement être un jeton Discord valide.

Les plafonds de requêtes — posés dans l'extracteur d'identité lui-même, donc un endpoint authentifié ajouté demain est couvert sans qu'on écrive une ligne. Le proxy de citations en a un second, plus serré, parce qu'il consomme un quota mensuel partagé. Un dépassement répond explicitement, il ne coupe pas la connexion en silence.

Les en-têtes de réponse — politique de sécurité de contenu, refus du reniflage de type, pas de référent. Avec une exception assumée et nommée — l'encadrement par Discord **doit** rester autorisé, l'Activity n'étant qu'une iframe ; durcir ce point-là rendrait le jeu invisible.

Les entrées venues du réseau — le pseudo est tronqué, le hash d'avatar validé et jeté s'il ne correspond pas au format attendu. On ne transporte jamais d'URL d'avatar — une URL fournie par un client serait une adresse arbitraire chargée dans le navigateur des sept autres.

La frontière de confiance de la course — le sujet du récit ci-dessous, et de la section 6.6.

RÉCIT Un « j'ai fini » partiel gagnait la course

Problème Le message de fin de course était cru sur parole. Trois caractères tapés très vite suffisaient donc à annoncer une arrivée : la durée était minuscule, la vitesse calculée absurde — de l'ordre de 800 mots par minute — et le joueur prenait la première place.

Décision Rejouer le journal de frappe côté serveur contre **son** texte et vérifier qu'il atteint réellement la fin, avant d'enregistrer quoi que ce soit. Une fin non confirmée n'est pas rejetée en silence : elle est enregistrée comme un abandon, le seul verdict à la fois vrai et débloquent pour les autres.

Résultat Ce n'était pas un correctif isolé mais l'entrée d'une frontière de confiance entière à reconstruire, poursuivie sur plusieurs issues : quels champs le client a-t-il le droit d'affirmer, lesquels ne font qu'**arrêter** quelque chose, et lesquels le serveur possède seul. La règle qui en est sortie tient en une phrase — une déclaration du client peut arrêter une course, jamais la gagner (issues 160, 163, 164, puis 186).

5.10 Le pipeline d'assets et la Rich Presence

Les assets visuels du projet ne sont pas des images dessinées dans un éditeur : ce sont des **programmes**. Une bibliothèque de composants vectoriels — la voiture, le clavier, le sol de lave, les feux de départ, le wordmark — est écrite en Typst, et chaque asset est un petit fichier qui l'importe, pose sa page et compose sa scène. Un script régénère l'ensemble d'une commande.

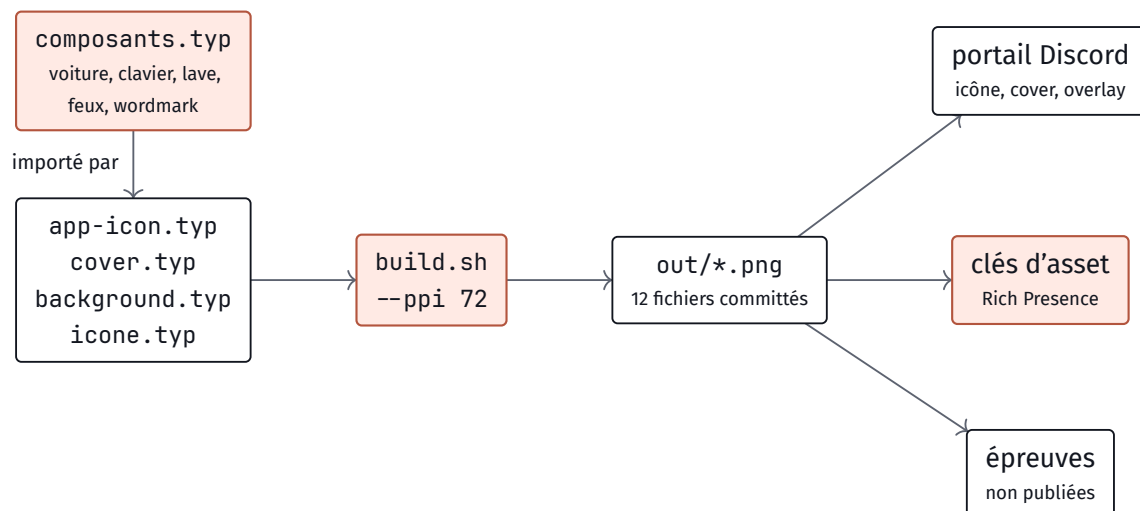


Figure 20. – Le pipeline d'assets. Le point à retenir est à droite : **le nom du fichier EST la clé d'asset** que le code demande à Discord. Renommer un PNG ne casse rien à la compilation — l'état correspondant perd simplement son visuel, silencieusement.

Deux règles ont été posées d'emblée, et elles se justifient l'une l'autre. La première : la page est déclarée en points et l'export forcé à 72 pixels par pouce, ce qui fait qu'un point vaut exactement un pixel — sans quoi le réglage par défaut livre le double des dimensions demandées, ce qui ne se remarque qu'au téléversement. La seconde : le générateur pseudo-aléatoire qui dessine la croûte de basalte du sol de lave est **déterministe**. Deux exécutions du script produisent des PNG identiques au bit près ; sans cela, chaque régénération salirait le diff des images committées.



Figure 21. – La voiture, composant central. Une seule silhouette fermée plutôt qu'un assemblage de rectangles : à 48 pixels dans l'étagère Discord, il ne reste que le contour, et un contour unique y survit là où une pile de formes se brouille.



Figure 22. – L'épreuve de contrôle, agrandie ici pour être lisible : elle ne fait que 48 pixels de côté. C'est à cette taille que l'icône apparaît dans l'étagère Discord, et c'est elle qui a décidé de chaque détail — le pavillon épais, les trois disques par roue, le nez biseauté.

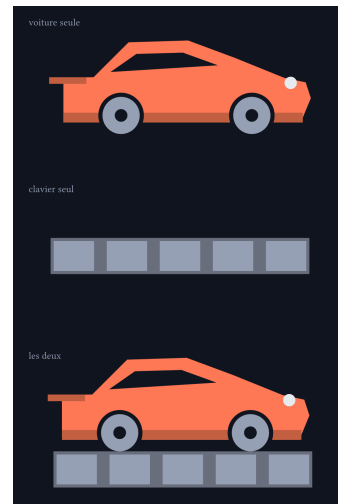


Figure 23. – Voiture et clavier partagent leur ligne de sol : la voiture roule littéralement sur les touches, sans aucune translation à écrire. C'est la métaphore du jeu — taper est ce qui la fait avancer.

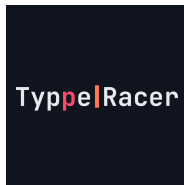


Figure 24. – Icône d'application.

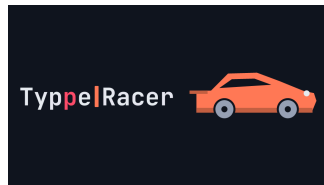


Figure 25. – Cover de l'étagère.

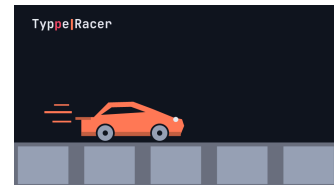


Figure 26. – Overlay de la grille.

Côté Rich Presence, le jeu pousse à Discord un état à chaque transition — menu, entraînement, salon, course — et Discord l'affiche aux autres membres du salon vocal. Le grand visuel change avec l'état, le petit reste l'icône de l'application. Un détail a été tranché en cours de route : dans un salon d'attente, c'est le visuel du **Mode de jeu choisi** qui s'affiche, pas une image d'attente générique — ce qu'on est sur le point de jouer intéresse davantage le lecteur que le fait qu'on attende. L'image « salon » reste donc dans le dossier, sans emploi.



Figure 27. – menu



Figure 28. – practice



Figure 29. – race



Figure 30. – floor-is-lava



Figure 31. – spam



Figure 32. – Lobby — produit, plus envoyé

5.11 Le graphe de résultats, sans bibliothèque

Le graphe seconde par seconde de l'écran de résultats reposait au départ sur une bibliothèque de graphiques. Elle a été retirée du projet et remplacée par un SVG construit à la main. Le raisonnement n'était pas la performance mais la proportion : le projet affiche **une** courbe, avec deux séries et une échelle fixée par le domaine. Le coût d'une dépendance qui sait tout dessiner — son poids dans le bundle, ses avis de sécurité à surveiller, sa version majeure à suivre — n'était pas payé par ce qu'on en utilisait.

5.12 La chaîne d'intégration et le déploiement

Trois travaux, dont deux tournent à chaque poussée et le troisième seulement sur la branche principale.

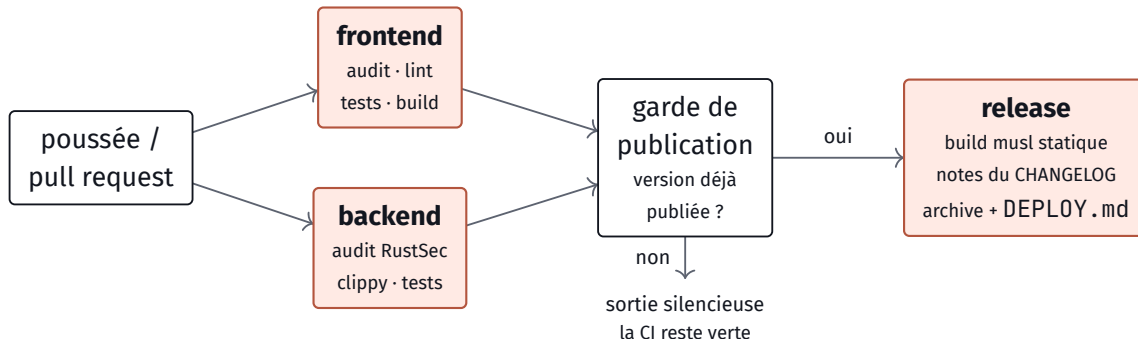


Figure 33. – La chaîne d'intégration. Le travail de publication **dépend** des deux autres, ce qui rend mécaniquement impossible de publier avec un test rouge — et c'est la raison pour laquelle il vit dans le même fichier plutôt que dans un workflow séparé.

La garde de publication lit la version du manifeste et la compare aux versions déjà publiées. Elle ne lit pas les étiquettes de version posées sur le dépôt : pousser une branche et pousser une étiquette sont deux événements sans ordre garanti entre eux, et une garde fondée là-dessus raterait une publication une fois sur deux, silencieusement.

La branche principale bouge pour deux raisons — de vraies versions, mais aussi des fusions de section et des poussées de documentation. Sur ces dernières, la version n'a pas changé : le travail sort alors sans rien faire et sans échouer. Le faire échouer en rouge apprendrait à ignorer la couleur de la CI.

RÉCIT Les gardes du travail de publication

Problème Le premier jet demandait à l'outil GitHub si la version était déjà publiée, **à l'intérieur d'une condition**. Deux pièges s'y superposaient. Le premier : dans une condition shell, l'arrêt automatique sur

	erreur est suspendu — une panne réseau, un quota ou un souci d'authentification se lisait donc « pas de release », et la publication partait. Le second : le titre de la version venait du fichier de notes et était interpolé directement dans la ligne de commande, où une paire de backquotes se serait exécutée . Ce n'était pas théorique : le fichier de notes du projet donne lui-même en exemple un titre contenant des backquotes.
Décision	Sortir l'appel réseau de la condition pour qu'un échec fasse honnêtement échouer le travail, lister les versions publiées plutôt que d'interroger l'une d'elles — un code de retour non nul ne distingue pas un « absent » d'une panne — et faire passer le titre par l'environnement, où aucun shell ne le relit.
Résultat	Deux classes de bugs qui ne se seraient manifestées qu'en production : une publication de travers un jour de panne réseau, et une exécution de commande arbitraire déclenchée par un fichier de notes. Aucune des deux n'aurait été attrapée par un test.

EN CLAIR Un binaire « statique »

Un programme compilé va normalement chercher au démarrage des bibliothèques déjà installées sur la machine. Si celle-ci en a une version plus ancienne que la machine qui a compilé, il refuse de démarrer — et on ne s'en aperçoit qu'au déploiement. Un binaire statique emporte tout avec lui : c'est un seul fichier qu'on dépose sur n'importe quel Linux et qui démarre. Le projet peut se le permettre parce qu'il n'utilise aucune bibliothèque système — sa base de données est compilée dedans, et son chiffrement est écrit en Rust.

L'archive publiée contient le binaire, le build du frontend, et une notice de déploiement générée. Cette notice insiste sur un point : le serveur écoute **volontairement** sur la boucle locale et non sur le réseau. Il est injoignable depuis une autre machine, parce que le mode de déploiement du projet est un tunnel qui tourne sur le même hôte. Exposer directement sur Internet demande un proxy inverse devant, et c'est un choix à poser explicitement.

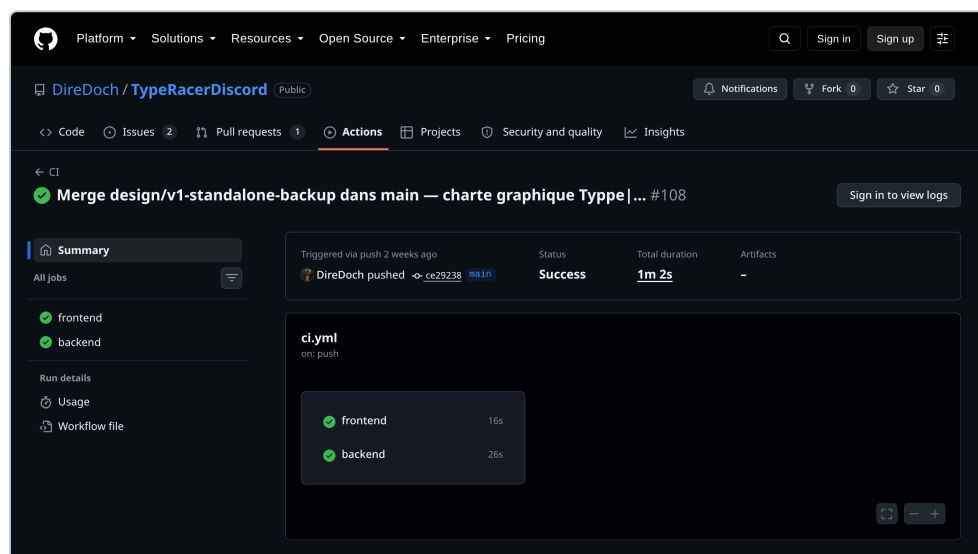


Figure 34. – Une poussée sur la branche principale, telle qu'elle s'affiche réellement : frontend et backend au vert, et **rien d'autre**. Le troisième travail est bien déclaré dans le fichier de workflow, mais aucune exécution du dépôt ne l'a jamais montré — voir section 7. Le champ « Artifacts » vide raconte la même chose.

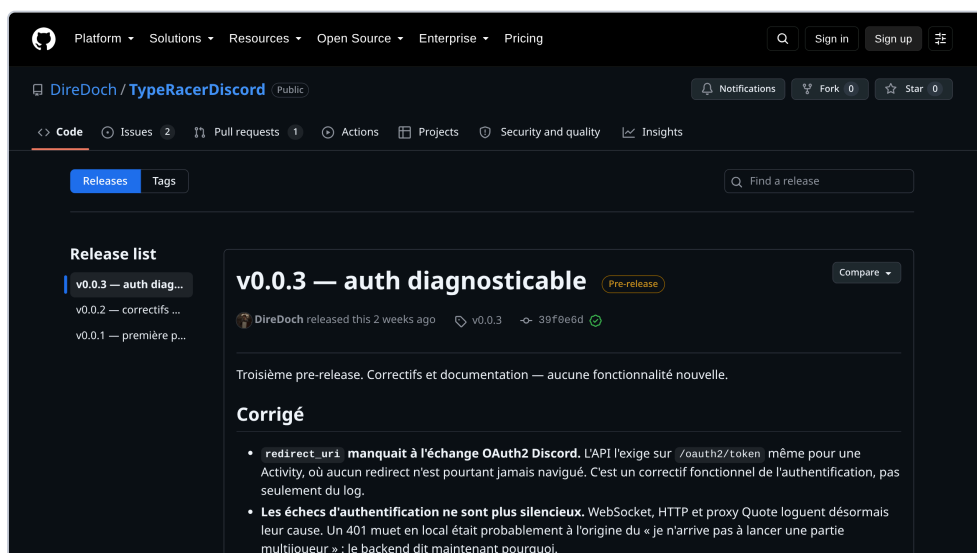


Figure 35. – Une version publiée. Les notes viennent du fichier de changements du dépôt et non des titres de commits — une version parle au joueur, et cela se voit. L'archive, elle, n'y est pas : les trois versions publiées sont antérieures au travail qui devait la produire.

5.13 Méthode : le modèle de domaine fait autorité sur le nommage

Le projet tient un glossaire de domaine à la racine, et ce glossaire n'est pas documentaire : il fait **autorité**. Chaque terme y porte non seulement sa définition mais la liste des mots interdits à sa place — « Race » interdit Match, Duel, Course ; « Abandon » interdit Quit, DNF. Le code, les issues et les messages d'interface s'y conforment.

L'intérêt s'est révélé sur les cas limites, pas sur les cas courants. Nommer « Brûlé », « Devancé » et « Échec » comme trois états **distincts** d'abandon — au lieu d'un seul état « perdu » à nuancer par un drapeau — a forcé à répondre à la question qui décide vraiment de leur comportement : **qui** a mis fin à ce Run ? Le joueur, sa propre faute, ou la comparaison aux autres ? Les cinq sorties du diagramme de la section 5.3 sont sorties de là, et le type qui les porte les rend exhaustives par construction : oublier d'en traiter une ne compile pas.

5.14 Méthode : la parité TypeScript ↔ Rust prouvée par vecteurs

L'algorithme de score existe deux fois : en TypeScript, où il est la **référence**, et en Rust, où il est autoritaire. Ce n'est pas une duplication accidentelle — le navigateur en a besoin pour le compteur en direct, le serveur en a besoin pour le verdict. Aucun générateur de types ne relie les deux : le miroir est **manuel**, et chaque fichier porte en en-tête le nom de son homologue.

RÉCIT Le miroir avait dérivé, sans filet

- Problème** Le décompte avant le départ était réglable, et le réglage semblait fonctionner : la valeur se choisissait, se rediffusait, s'affichait. Mais les deux côtés n'étaient plus d'accord sur la valeur par défaut — le décompte disait sept secondes d'un côté et cinq de l'autre. Cliquer sur le réglage ne faisait donc **rien** de visible, et rien ne signalait l'écart : ni erreur, ni test rouge, ni avertissement.
- Décision** Extraire des **vecteurs de test partagés** — des jeux d'entrée avec leur résultat attendu, dans un dossier à la racine — et faire lire ces mêmes fichiers par les deux suites de tests, celle de TypeScript et celle de Rust.
- Résultat** Une divergence entre les deux langages devient un test rouge dans l'un des deux, au lieu d'un comportement silencieusement faux. C'est le seul filet possible pour un miroir manuel — et le prix est modeste : les vecteurs sont des données, pas du code.

EN CLAIR Un « miroir manuel » entre deux langages

La même règle de calcul doit exister dans le navigateur et sur le serveur, qui ne parlent pas le même langage. On peut demander à un outil de traduire automatiquement l'un vers l'autre — c'est lourd, et l'outil devient une pièce du projet à entretenir. Ou on peut écrire les deux à la main et vérifier qu'ils disent la même chose, en leur donnant les mêmes questions et en comparant leurs réponses. Le projet a choisi la seconde option ; les « vecteurs » sont ces questions communes.

5.15 Méthode : les décisions d'architecture consignées

Dix-neuf décisions sont consignées, une par fichier, dans un dossier dédié. Chacune répond à une question qui ne se lit pas depuis le code seul : pourquoi une Race n'a pas de Mode, pourquoi le texte cible est persisté tel quel plutôt que régénéré depuis sa graine, pourquoi le message de fin porte les résultats et pas seulement l'ordre. La table complète est en annexe.

Leur valeur réelle est apparue tard. Quand un fichier de réseau a atteint trois mille trois cent soixante-et-onze lignes — le fil réseau, le salon et le moteur de course entassés au même endroit — la décision de le découper n'a pas eu à être redécouverte : les frontières étaient déjà nommées dans les décisions existantes, il ne restait qu'à les rendre visibles dans l'arborescence.

RÉCIT Un fichier de 3 371 lignes

Problème Tout le multijoueur vivait dans un seul fichier : la gestion du socket, l'état du salon, les réglages, et le moteur de course avec ses trois Modes de jeu. Ajouter un Mode de jeu revenait à toucher au même fichier que corriger une déconnexion — chaque chantier multijoueur entraînait en collision avec les autres.

Décision Découper selon les frontières que les décisions d'architecture nommaient déjà : le Mode de jeu comme table déclarée, les Réglages de salon avec un contrat de retour explicite, le moteur de course séparé du transport.

Résultat Le découpage n'a pas été une réécriture mais une **révélation** : les frontières existaient dans le vocabulaire du domaine avant d'exister dans les fichiers. C'est l'argument le plus concret que le projet ait produit en faveur des deux chantiers de méthode précédents (issue 205).

6 Documentation technique

Troisième et dernier point de vue : le système **à l'état final**. Les sections précédentes racontaient une construction ; celle-ci décrit ce qui tient debout une fois les échafaudages retirés. Elle est faite pour être lue dans le désordre, section par section, par quelqu'un qui cherche un point précis.

6.1 Arborescence du projet

Voici les entrées de racine qui portent la structure — le reste sont des fichiers de projet ordinaires. La séparation qui compte est celle des deux premières : `backend/` et `frontend/` sont deux programmes distincts, écrits dans deux langages, et **tout le reste du document** découle de la façon dont ils se parlent.

<code>backend/</code>	le serveur Rust : API HTTP, WebSocket, SQLite — et le service du build frontend, ce qui en fait l'origine unique
<code>src/</code>	détaillé plus bas
<code>migrations/</code>	six fichiers SQL, appliqués au démarrage
<code>frontend/</code>	le jeu lui-même, en TypeScript sans cadriciel : le DOM est manipulé directement
<code>src/</code>	détaillé plus bas
<code>index.html</code>	la coquille servie dans l'iframe
<code>design/</code>	les assets visuels, écrits en Typst et exportés en PNG
<code>composants.typ</code>	la voiture, le clavier, la lave, le wordmark
<code>build.sh</code>	régénère tout <code>out/</code> en une commande (section 5.10)
<code>Docs/</code>	ce document, les ADR, le contrat d'API, les procédures d'agent
<code>documentation.typ</code>	la source de ce PDF
<code>adr/</code>	les 19 décisions, une par fichier
<code>test-vectors/</code>	les vecteurs de parité, lus par les deux suites de tests (section 5.14)
<code>Contexte/</code>	les notes de cadrage d'origine, gardées telles quelles — de l'histoire, pas de la référence
<code>CONTEXT.md</code>	le glossaire de domaine. Autorité sur le nommage, pas documentation d'appoint
<code>PONYTAIL-DEBT.md</code>	le registre des raccourcis délibérés, chacun avec son plafond et son chemin de sortie

Trois fichiers de racine ne sont pas là par goût de la paperasse : `LICENSE`, `TERMS.md` et `PRIVACY.md` sont **exigés** par le portail développeur Discord pour qu'une Activity puisse être publiée. Ils sont listés en section 8.

`backend/src/` — le serveur

Deux sous-dossiers portent la structure. `domain/` est le domaine pur — aucune entrée-sortie, aucun socket, aucune base : ce sont les fonctions qui **décident**. `ws/` est le multijoueur, découpé par l'issue 205 en modules qui portent chacun une frontière nommée — c'est le fichier de 3 371 lignes du récit précédent.

<code>main.rs</code>	le routeur Axum, les extracteurs d'authentification, les en-têtes de sécurité, l'écoute sur la boucle locale
<code>discord.rs</code>	l'identité et OAuth2 : échange du code, résolution du <code>player_id</code> , vérification de l'application émettrice
<code>store.rs</code>	la persistance SQLite. Le PB n'y a pas de table : il se dérive
<code>quote.rs</code>	le proxy vers l'API de citations. La clé ne quitte jamais le serveur
<code>rate_limit.rs</code>	les plafonds de requêtes. Une seule table pour tous les seaux
<code>domain/</code>	le domaine pur, autoritaire — miroir Rust de <code>frontend/src/core/</code>
<code>mod.rs</code>	la déclaration du miroir : quel fichier Rust reflète quel fichier TypeScript
<code>types.rs</code>	les types de domaine partagés
<code>replay.rs</code>	le recompute autoritaire du Scoreboard. Le cœur de la frontière de confiance
<code>difficulty.rs</code>	l'échec Expert/Master, rejoué serveur avant d'être cru
<code>text_gen.rs</code>	la génération seedée. Même graine, même texte que le navigateur
<code>spam.rs</code>	le comptage des répétitions verrouillées du Mode de jeu Spam
<code>analysis.rs</code>	le moteur Weak spot : quelles touches coûtent le plus cher
<code>ws/</code>	le multijoueur — sockets, Rooms, moteur de course
<code>mod.rs</code>	le fil : l'état partagé des Rooms, la boucle socket, le dispatch
<code>protocol.rs</code>	les 29 messages — miroir de <code>core/net.ts</code>
<code>room.rs</code>	la Room comme lieu : entrer, sortir, l'hôte, le Code de partie
<code>room_setting.rs</code>	le Réglage de salon, avec son contrat de retour explicite (ADR 0017)
<code>race_engine.rs</code>	le déroulé d'une course, du <code>StartRace</code> au <code>RaceOver</code>
<code>game_mode.rs</code>	le seam du Mode de jeu : une table déclarée, pas un <code>match</code> recopié
<code>tests.rs</code>	les tests du fil, de la Room et du moteur

frontend/src/ — le jeu

Même principe, une frontière de plus. `core/` est le domaine pur du client : il ne touche jamais au DOM et c'est ce qui le rend testable sans navigateur. `ui/` dessine, et ne décide de rien qui compte.

<code>main.ts</code>	l'amorçage : le routage entre écrans, le bandeau d'erreurs in-iframe
<code>api.ts</code>	la frontière HTTP. Toutes les requêtes passent par ici — c'est ce qui rend le préfixe <code>/ .proxy</code> gérable en un point
<code>discord.ts</code>	le handshake d'identité côté client, et <code>proxyBase()</code>
<code>live-stats.ts</code>	le compteur de vitesse en direct. Non autoritaire, et assumé comme tel
<code>style.css</code>	la feuille de style, unique. 2 312 lignes, et la palette dont ce PDF hérite
<code>content/lessons.json</code>	les 100 leçons. Une donnée , pas du code
<code>core/</code>	le domaine pur du client — ni DOM, ni réseau, donc testable sans navigateur
<code>types.ts</code>	les types de domaine. La référence dont Rust est le miroir
<code>net.ts</code>	le transport WebSocket et les 29 messages, côté client
<code>race-state.ts</code>	l'état d'une Race : une transition pure sur chaque <code>ServerEvent</code>
<code>run-session.ts</code>	une Run tapée : horloge, contrôleur, journal, Difficulté
<code>clock.ts</code>	l'origine du temps. <code>t=0</code> est la première frappe
<code>countdown.ts</code>	le décompte annulable, de durée réglable
<code>difficulty.ts</code>	l'échec Expert/Master. La référence de l'algorithme
<code>learn.ts</code>	le moteur du cursus : le barème par tranches, le déverrouillage
<code>preferences.ts</code>	les Preferences. Ne quittent jamais l'appareil
<code>sound.ts</code>	les effets sonores
<code>fps.ts</code>	la limite d'images par seconde de l'animation
<code>speed-unit.ts</code>	la conversion entre unités de vitesse
<code>stats/scoreboard.ts</code>	le calcul du Scoreboard. La référence , portée en Rust
<code>text-gen/</code>	le PRNG seedé, la liste de mots, la ponctuation, les nombres, les drills
<code>input/</code>	les contrôleurs de saisie, derrière une interface commune
<code>ui/</code>	le rendu — dessine, et ne décide de rien qui compte
<code>chrome.ts</code>	l'habillage qui ne change jamais d'écran
<code>menu.ts</code>	le menu principal et le hub de navigation
<code>practice.ts</code>	l'écran de Practice
<code>race.ts</code>	l'écran de Race : la piste, les voitures, la vitesse en direct
<code>lobby-rows.ts</code>	les Réglages de salon du lobby, déclarés puis rendus
<code>podium.ts</code>	le podium et le Gap
<code>potg.ts</code>	Play of the Game : le duel au ralenti sur horloge partagée
<code>results.ts</code>	l'écran de résultats solo
<code>chart.ts</code>	le graphe seconde par seconde, en SVG écrit à la main (section 5.11)
<code>replay.ts</code>	la relecture d'un Run depuis son Keystroke log
<code>typing-zone.ts</code>	la zone de frappe — la fenêtre de trois lignes
<code>learn.ts</code>	l'écran « Apprendre » : la liste et l'exercice
<code>settings.ts</code>	l'écran Paramètres
<code>history.ts</code>	l'historique des Runs
<code>weak-spots.ts</code>	le rendu d'une analyse de touches faibles
<code>guide.ts</code>	le guide « Comment jouer », affiché d'office la 1 ^{re} fois
<code>mode-labels.ts</code>	les libellés français des Modes
<code>info-bubble.ts</code>	l'icône « i » et sa bulle d'explication

6.2 Architecture générale et frontière client/serveur

Le serveur Rust a **deux** rôles, et c'est la contrainte Discord qui l'a décidé : une Activity ne reçoit qu'un seul URL Mapping, donc une seule origine. Un serveur pour l'interface et un autre pour l'API auraient demandé deux adresses. Le binaire sert donc le build Vite en fichiers statiques **et** expose l'API sur la même origine.

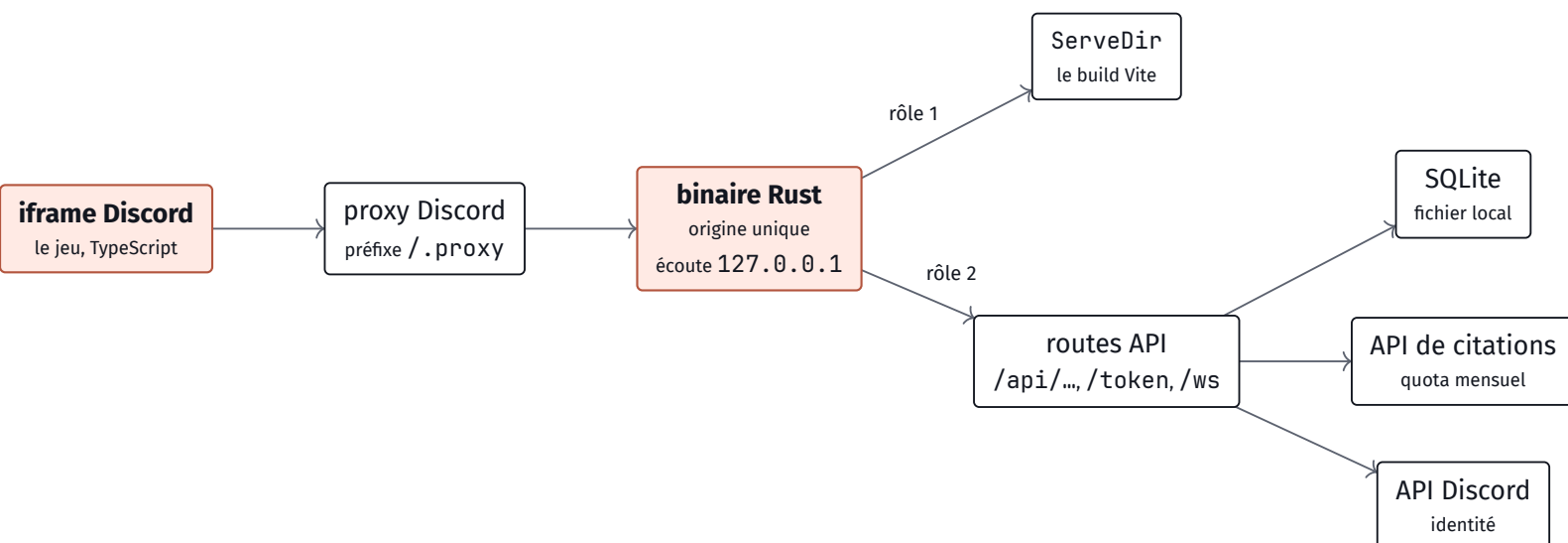


Figure 36. – L'architecture globale. Le binaire n'écoute **pas** sur le réseau mais sur la boucle locale : c'est un tunnel qui tourne sur le même hôte qui l'expose. Les trois flèches de droite sont les seules sorties du serveur — la base est un fichier, pas un service.

Côté modules, la frontière qui structure tout est celle du **domaine pur**. Des deux côtés de la ligne réseau, on trouve le même dessin : un noyau qui ne connaît ni le DOM ni les sockets, et une couche qui l'utilise.

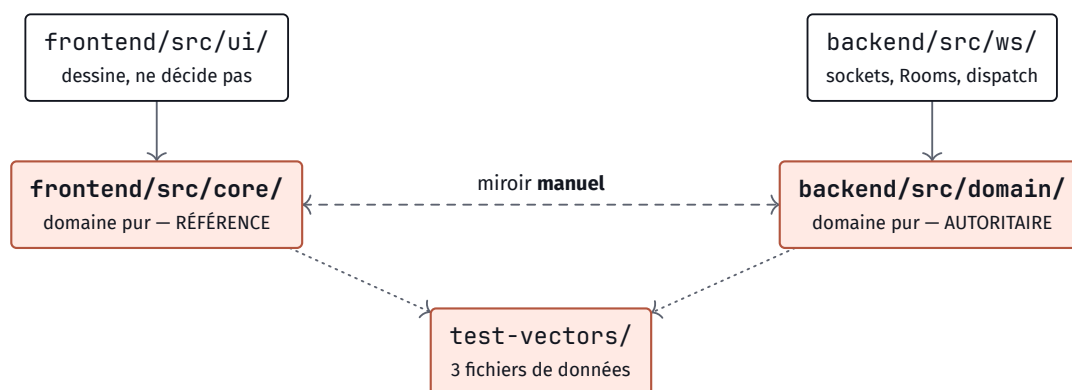


Figure 37. – La carte des modules. La ligne pointillée horizontale est le miroir manuel : aucun générateur ne relie les deux domaines, chaque fichier nomme son homologue en en-tête. Les deux pointillés du bas sont le seul filet — les mêmes fichiers de vecteurs, lus par vitest et par cargo test.

Ce que la frontière réseau tranche, en une phrase : le client **possède** ce que le joueur voit et règle pour lui-même ; le serveur possède tout ce qui classe, compte ou persiste. Les deux détails qui font la différence : le client ne reçoit jamais la graine sans le texte qui en découle, et il ne renvoie jamais le texte au serveur.

6.3 Le protocole WebSocket

C'est la section la plus dense du document, et celle qui décrit le cœur du projet : ce qui se passe entre huit navigateurs et un serveur pendant qu'une course se joue. Le schéma ci-dessous en pose la forme d'ensemble — qui est connecté à quoi, dans quel sens circule quoi, et surtout à **qui** chaque message est adressé.



Figure 38. – Le flux réseau d'une Room. Chaque joueur tient sa propre connexion : une Room est un état **serveur**, pas un tuyau partagé — c'est ce qui permet à un message de ne viser qu'un seul destinataire. En bas, les trois portées : le même socket sert la diffusion générale, le message privé et la réponse au seul demandeur, et c'est le message qui décide, jamais le client.

Pourquoi un WebSocket, et pas des requêtes

Le critère n'est pas la vitesse mais la **direction**. Trois messages du serveur n'ont aucune requête client qui puisse leur correspondre : RaceStart tombe quand l'hôte lance la partie, PlayerBurned quand le métronome de Floor is lava bat, SpamStop quand un plafond de temps expire. Aucun de ces trois-là n'est la **réponse** à quoi que ce soit. Avec des requêtes, le client devrait redemander « du nouveau ? » en boucle, et un décompte partagé à la seconde près exigerait de redemander plusieurs fois par seconde, à huit joueurs, pendant toute la partie.

EN CLAIR WebSocket contre requête HTTP

Une requête ordinaire, c'est le client qui pose une question et le serveur qui répond : le serveur ne peut jamais parler le premier. Pour savoir si la course a démarré, le navigateur devrait redemander sans arrêt, et l'information arriverait toujours avec un peu de retard. Un WebSocket est une ligne qui reste **ouverte** dans les deux sens pendant toute la partie : le serveur envoie quand il a quelque chose à dire, et personne n'a besoin de redemander.

Les 29 messages

Vingt messages montent du client, neuf descendent du serveur. Ils sont sérialisés en JSON avec une étiquette de type, et la même liste existe en TypeScript dans `core/net.ts` — c’est encore le miroir manuel.

Présentées en cartes plutôt qu'en tableau, et pour une raison de lecture : on ne parcourt pas ces vingt-neuf lignes de haut en bas, on en cherche **une**. En tableau, l'œil glisse ; encadré, chaque message existe séparément. La mention en petit à droite de chaque carte est le droit associé — c'est elle, et pas le message, qui décide si le serveur écoute.

JoinChannel	tous	CreateRoom	tous
Rejoindre la Room du salon vocal, en la créant si besoin. La clé vient du SDK Discord : elle ne peut pas être mal tapée.		Ouvrir une Room neuve. Le serveur tire le Code de partie et le renvoie dans le <code>RoomState</code> qui suit.	
JoinCode	tous	StartRace	hôte
		Lancer la course. Le seul message dont l'effet est irréversible pour toute la Room.	

Rejoindre par Code. Ne crée jamais : un code vient d'un clavier, et créer sur une faute de frappe enfermerait le joueur seul.	
SetTextSource hôte · hors course Régler d'où vient le texte. Déclenche la régénération.	SetGameMode hôte · hors course Normal, Floor is lava ou Spam. Régénère le texte : chaque mode impose le sien.
SetMaxPlayers hôte · hors course Taille maximale (2–8). Ne porte que sur les arrivées : personne n'est expulsé.	SetCountdown hôte · hors course Durée du décompte : 3, 5, 7 ou 10 s.
SetDifficulty hôte · hors course Normal ou Master. Expert n'est pas un Réglage de salon : sa condition de déclenchement est inatteignable en Race.	SetReadyCheck hôte · hors course Activer le ready-check. Réinitialise les prêts déjà marqués.
SetLavaInterval hôte · hors course Intervalle d'élimination, parmi 5/10/15/20 s. Inerte hors Floor is lava.	SetSpamWord hôte · hors course Le mot répété. Validé serveur : non vide, sans espace, 20 caractères au plus — un espace ferait de « un mot répété » plusieurs mots cibles.
SetSpamThreshold hôte · hors course Le seuil de répétitions qui gagne, parmi 10/20/30/50.	SetSpamTimeCap hôte · hors course Le plafond de temps de Spam, 60 s au plus.
SetReady tous · hors course Se marquer prêt. Le seul réglage que tout le monde peut poser — il ne porte que sur soi.	Progress partants Position de frappe, et le compte de répétitions sous Spam. Déclaratif : sert au rendu des voitures, jamais au classement.
Finish partants La soumission : le journal de frappe brut et l'instant de fin. Ni le texte ni la durée de référence — le serveur les possède.	Forfeit partants Abandon volontaire. Le joueur reste au lobby pour la suivante.
Fail partants Échec Master détecté localement. Le serveur rejoue le journal pour le confirmer avant de l'enregistrer.	LeaveRoom tous Quitter la Room. Produit exactement le même enregistrement qu'une déconnexion — un seul chemin de code.

Figure 2. – Les vingt messages `ClientEvent`. La mention de droite n'est pas une convention d'interface : un message hors droits est ignoré côté serveur, en silence. Un client modifié ne gagne rien à l'envoyer quand même.

Côté descendant, les neuf messages se distinguent moins par leur contenu que par leur **portée** — à qui ils sont adressés. Les cartes le portent en tête, parce que c'est le premier fait à connaître sur un message qui arrive.

RoomState toute la Room L'état complet du salon : les présents avec leur identité d'affichage, l'hôte, la graine, le texte, le Code, et les dix réglages. Rediffusé à chaque changement.	RaceStart toute la Room Le top de départ : un instant absolu partagé, qui cale les horloges locales de tout le monde sur le même $t=0$.
PlayerProgress toute la Room La position d'un joueur, pour le rendu de sa voiture. Non autoritaire, et c'est assumé.	PlayerFinished toute la Room Le scoreboard recalculé d'un arrivant. Distingue une vraie arrivée, un abandon et un échec Master.

RaceOver Le classement complet, avec les courbes de chacun et le duel de Play of the Game. L'ordre du tableau est le classement.	toute la Room	SpamStop La course Spam s'arrête. Un seul message pour les deux causes, et il ne désigne aucun vainqueur.	toute la Room
PlayerBurned Ce joueur vient de brûler. C'est ce message qui lui demande son journal et lui dit d'arrêter de taper.	un seul joueur	RoomNotFound Code inconnu. Le socket reste ouvert : le joueur corrige sans se reconnecter.	le socket demandeur
RoomFull Room déjà pleine. Même traitement : réponse directe, socket gardé.	le socket demandeur		

Figure 3. – Les neuf messages ServerEvent, rangés par portée décroissante. Ces trois portées sont ce que le schéma d'ouverture de cette section dessine, et la seule chose qu'un client ne peut pas deviner du message lui-même.

Les deux bouts du fil

Le même message traverse deux langages. Voici les deux endroits où il est reçu — condensés, mais fidèles au dépôt.

Côté serveur, la boucle de lecture déséréalise et aiguille. Ce qu'il faut y voir : les onze réglages ne sont pas onze branches de logique, mais onze constructions d'une même variante RoomSetting passée à une seule fonction. C'est le seam de l'ADR 0017 — le jour où un douzième réglage arrive, le dispatch n'apprend rien de nouveau.

Le dispatch : une boucle, un match, une seule porte pour les réglages

backend/src/ws/mod.rs

```

while let Some(Ok(msg)) = receiver.next().await {
    let text = match msg {
        Message::Text(t) => t,
        Message::Close(_) => break,
        _ => continue,
    };
    match serde_json::from_str::<ClientEvent>(&text) {
        // Un Réglage de salon = une variante `RoomSetting` (#204).
        // Le dispatch ne sait plus lesquels demandent un texte neuf.
        Ok(ClientEvent::SetGameMode { mode }) => {
            let s = RoomSetting::GameMode(mode);
            apply_room_setting(&rooms, &key, &player_id, s, &quotes)
        }
        // PAS un Réglage de salon (ADR 0017) : chacun se marque prêt.
        Ok(ClientEvent::SetReady { ready }) => {
            set_ready(&rooms, &key, &player_id, ready)
        }
        Ok(ClientEvent::Progress { chars_done, reps }) => {
            let now = now_epoch_ms();
            relay_progress(&rooms, &key, &player_id, chars_done, reps, now)
        }
        Ok(ClientEvent::Finish { keystrokes, ended_at_ms }) => {
            // `finish_race` REJOUÉ le log contre le texte du serveur.
            if finish_race(&rooms, &key, &player_id, keystrokes,
                           ended_at_ms, &pool) {
                spawn_refresh_text(rooms.clone(), key.clone(), quotes.clone());
            }
        }
        Ok(ClientEvent::LeaveRoom) => break,
        Ok(_) => {} // jointure en double : ignorée
        Err(e) => eprintln!("WS {player_id} : illisible ({e})"),
    }
}

```

Côté client, la réception ne fait rien d'autre que désérialiser et confier l'événement à une fonction **pure**. C'est cette pureté qui rend l'état d'une Race testable sans navigateur et sans serveur : `applyServerEvent` prend un état et un message, et rend un état.

La réception : un transport de trois lignes, et une transition pure

frontend/src/core/net.ts · core/race-state.ts

```
// net.ts — le transport ne décide de rien, il ne fait que porter.
const proto = location.protocol === "https:" ? "wss" : "ws";
const url = `${proto}://${location.host}${basePath}/ws`;
this.ws = new WebSocket(`${url}?token=${encodeURIComponent(token)}`);
this.ws.onmessage = (m) => onEvent(JSON.parse(m.data) as ServerEvent);

// race-state.ts — état + message => état.
// Aucun DOM, aucun `fetch`, aucune horloge : une fonction pure.
export function applyServerEvent(
  state: RaceState,
  event: ServerEvent,
): RaceState {
  switch (event.type) {
    case "RaceStart":
      // Un seul décompte vivant : un second RaceStart est ignoré.
      if (state.phase === "countdown" || state.phase === "running") {
        return state;
      }
      return { ...state, phase: "countdown", racers: state.players.slice() };

    case "PlayerBurned":
      // Le seul message qui ne vise qu'un joueur : il demande son log.
      return advance(state, event.playerId, {
        kind: "burned",
        atMs: event.atMs,
      });

    case "RoomFull":
      return { ...state, phase: "failed", failure: ROOM_FULL };
  }
}
```

EN CLAIR Pourquoi « une fonction pure » change quelque chose

Une fonction pure ne regarde rien d'autre que ce qu'on lui donne, et ne touche à rien d'autre que ce qu'elle rend. Celle-ci reçoit l'état de la course et un message, et rend le nouvel état — elle ne lit pas l'heure, n'écrit pas à l'écran, n'ouvre aucune connexion. Conséquence pratique : pour la tester, il suffit de lui passer une suite de messages et de comparer le résultat. Pas de navigateur, pas de serveur, pas de course à jouer pour de vrai.

La frontière de confiance

Le partage est le même à chaque fois : le serveur possède la graine, le texte et $t=0$; le client possède ce qu'il a tapé. Un client peut donc **décrire** sa frappe, jamais son résultat.

INVARIANT Une déclaration du client peut arrêter une course, jamais la gagner

C'est la phrase sortie de l'issue 160, et la seule à retenir de toute cette section. Elle se décline en trois rangs, et tout message du protocole tombe dans l'un des trois.

affirmer son journal de frappe, son identité d'affichage, ses réglages s'il est l'hôte. Tout est assaini ou rejoué avant d'être utilisé — « affirmer » n'a jamais voulu dire « être cru ».

arrêter Progress.reps peut déclencher l'arrêt d'une course Spam, et une fin annoncée peut débloquent les autres. Ni l'un ni l'autre n'attribue une place. Un tricheur peut donc gâcher une partie ; il ne peut pas la remporter.

ne jamais toucher la graine, le texte cible, l'instant de départ, la durée de référence, le classement, le verdict de record. Rien de tout cela ne circule dans le sens client → serveur, à aucun moment.

Les trois portes d'entrée

Elles ont l'air interchangeables et ne le sont pas. JoinChannel crée à la volée, parce que sa clé est un identifiant de salon fourni par Discord : elle est authentique par construction. CreateRoom fait tirer un code par le serveur. JoinCode ne crée jamais, parce que sa clé vient d'un clavier humain : créer sur une faute de frappe donnerait une Room fantôme où le joueur attendrait seul, en croyant avoir rejoint ses amis.

Les garde-fous

Taille des messages — bornée à 256 Ko. Sans borne, la limite par défaut est de 64 Mio, et un Finish géant se recompute **sous le verrou global des Rooms** — une seule requête suffirait à figer toutes les parties du serveur.

Connexions — 20 par joueur et par minute. Ce sont les connexions qui sont comptées, pas les messages : chacune coûte une résolution d'identité auprès de Discord et une place dans la Room.

Identité — assainie à l'entrée. Le nom perd ses caractères de contrôle et est tronqué ; le hash d'avatar est **jeté** s'il sort de [0-9a-f_]. On ne transporte jamais d'URL d'avatar — une URL fournie par un client serait une adresse arbitraire chargée dans le navigateur des sept autres.

L'assainissement de l'identité ne protège pas d'une injection : le rendu échappe déjà le HTML. Il protège du fait qu'un nom démesuré dégrade l'écran **des autres**.

L'autorisation est la connexion

Le jeton est présenté une seule fois, dans l'URL du WebSocket, avant même que la connexion ne s'établisse — l'API WebSocket du navigateur ne permet pas d'en-tête Authorization. Une fois la connexion ouverte, elle **est** la preuve d'identité : il n'y a plus rien à vérifier message par message.

C'est ce qui permet à RaceOver de porter tous les résultats d'un coup, courbes comprises, sans aucun aller-retour (ADR 0010). L'alternative — un endpoint qui servirait le détail d'un joueur à la demande — buterait sur un mur : la composition d'une course vit en mémoire et meurt avec la Room, donc le serveur ne pourrait pas vérifier après coup que le demandeur en faisait partie. Sur le socket, la question ne se pose pas.

Enfin, dans l'iframe, l'adresse du socket passe par le préfixe /.proxy, comme toute autre requête. C'est la même fonction qui décide du préfixe pour les quatre points d'accès réseau du projet.

Une Race normale, de bout en bout

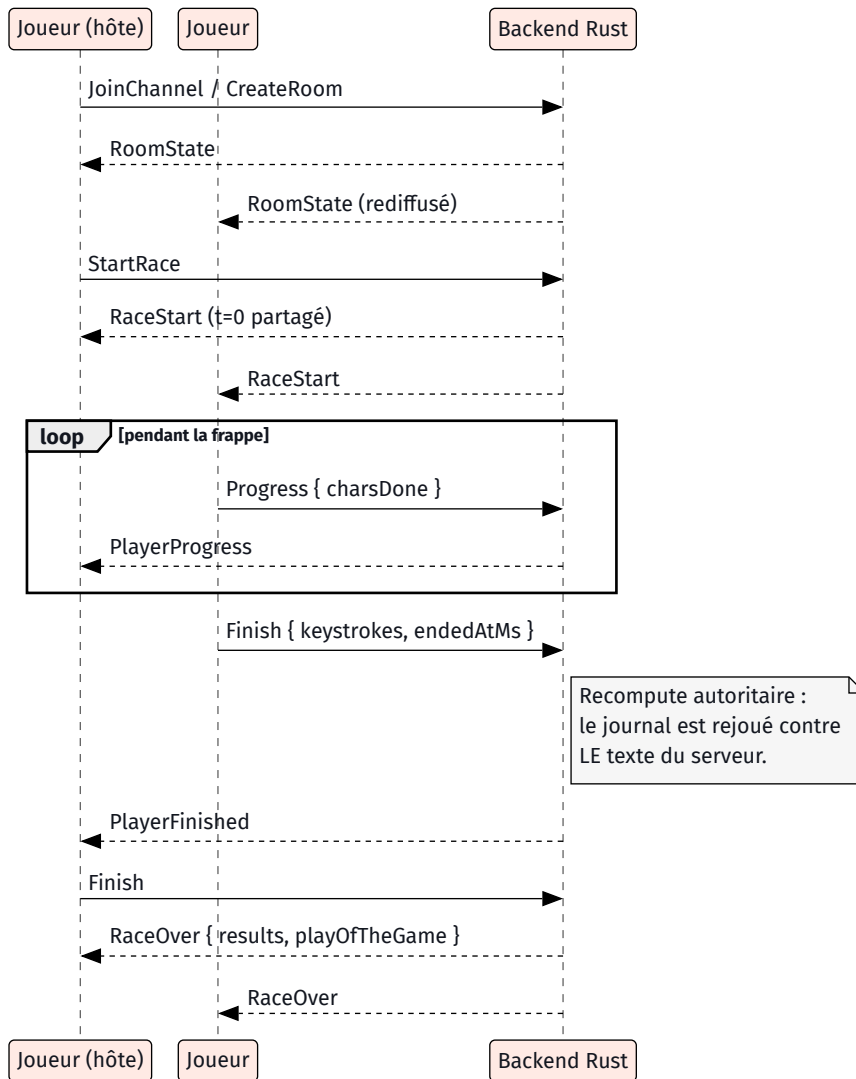


Figure 39. – Une Race normale. Le `RaceOver` ne part qu'une fois le **dernier** journal attendu arrivé — c'est pourquoi un abandon débloquent les autres au lieu de les faire patienter jusqu'au watchdog. Les deux `Finish` sont recomputés séparément ; le classement, lui, se décide en une seule fois.

Floor is lava

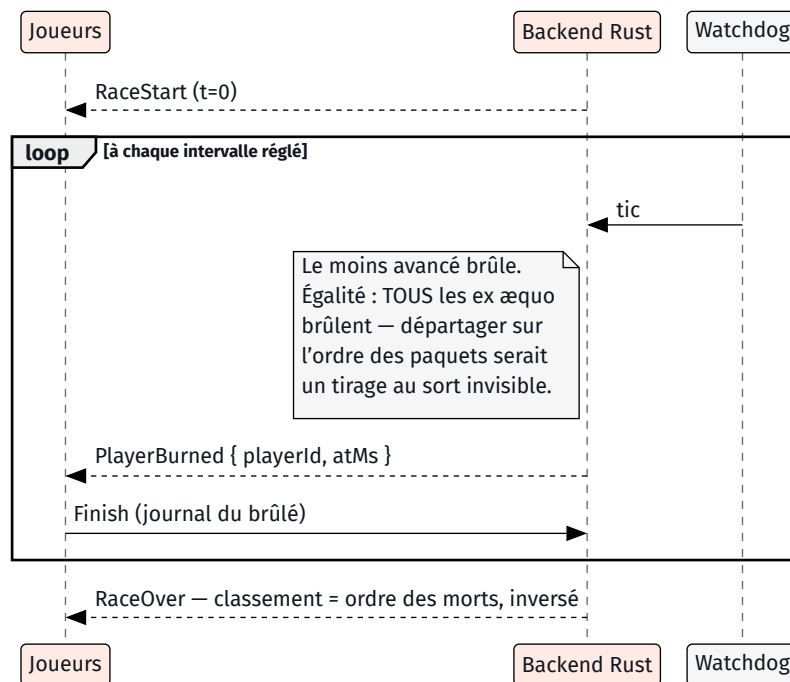


Figure 40. – Floor is lava. Le métronome part au **signal de départ partagé** et non à la création de la course : sinon le premier battement tomberait pendant le décompte, où personne n’a encore tapé — tout le monde est dernier, tout le monde brûle (issue 159). La course s’arrête à un vivant ; à zéro vivant, une égalité finale a emporté les deux derniers et il n’y a pas de vainqueur.

Le brûlé renvoie tout de même son journal, et le serveur le recompute : un joueur éliminé porte un **vrai score partiel**, calculé sur ce qu’il a eu le temps de taper. Ce score ne le classe jamais — seul l’instant de sa mort le classe — mais il s’affiche, et il sert à choisir le duel d’après-course.

6.4 Le contrat HTTP

Dix routes déclarées. Tout ce qui n’en atteint aucune part au service de fichiers statiques, puis à `index.html` : c’est ce qui fait qu’un serveur unique peut porter l’interface **et** l’API sans que le routage ait à trancher entre les deux. Les corps de requête et de réponse ne sont pas ici — ils vivent dans `Docs/API.md`, et les recopier garantirait que les deux versions divergent.

MÉTHODE	CHEMIN	RÔLE	AUTHENTIFICATION
GET	<code>/api/health</code>	Sonde de vie.	aucune
POST	<code>/token</code>	Échange le code OAuth2 contre un jeton d’accès. Le secret client ne quitte jamais le serveur.	aucune, plafond global
GET	<code>/api/quote</code>	Proxy vers l’API de citations. La clé est injectée côté serveur.	Bearer + plafond serré
POST	<code>/api/runs</code>	Soumission d’un Run solo : recompute, persistance, verdict de record.	Bearer
GET	<code>/api/runs/:id</code>	Un Run complet pour le Replay. 404 indistinctement : inconnu, à un autre, ou non rejouable.	Bearer
GET	<code>/api/runs/:id/analysis</code>	Les touches faibles d’un Run.	Bearer
GET	<code>/api/profile/analysis</code>	Les touches faibles sur les derniers Runs du joueur.	Bearer

MÉTHODE	CHEMIN	RÔLE	AUTHENTIFICATION
GET	/api/history	L'historique des Runs du joueur.	Bearer
GET POST	/api/learn/progress	Lire et avancer la progression du cursus. L'écriture conserve le maximum , jamais la dernière valeur reçue.	Bearer
GET	/ws	La Race entière. Jeton dans l'URL, faute d'en-tête possible.	?token= + plafond

Tableau 2. – Le contrat HTTP. Le 404 indistinct de /api/runs/:id est délibéré : distinguer « ce Run n'existe pas » de « ce Run n'est pas à vous » dirait à un curieux lesquels de ses identifiants devinés sont bons.

Un détail vaut d'être noté, parce qu'il se paie une fois et rapporte indéfiniment : le plafond de requêtes n'est pas posé dans chaque handler mais dans **l'extracteur d'identité** que tous les endpoints authentifiés traversent. Un endpoint ajouté demain est couvert sans qu'on écrive une ligne — et surtout sans qu'on puisse oublier de l'écrire.

6.5 Persistance

Deux tables, et le record personnel n'en a aucune.

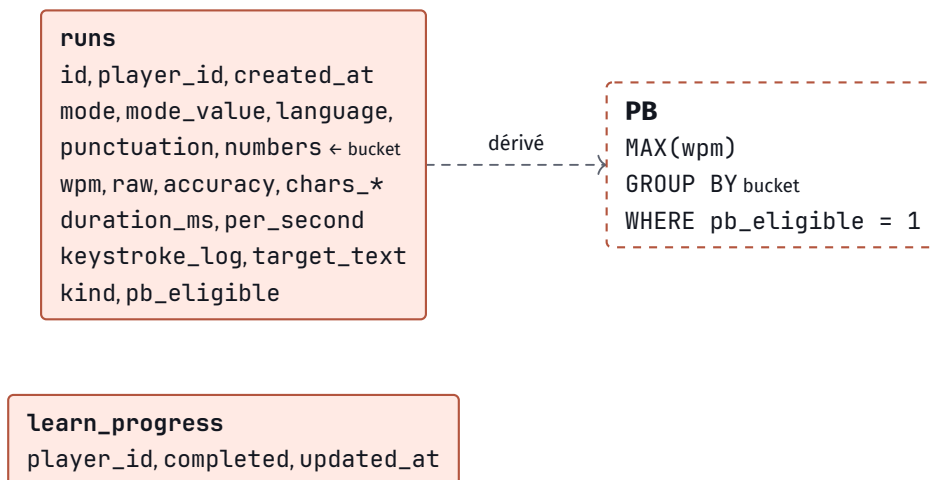


Figure 41. – Le schéma. Le PB est encadré en pointillés parce qu'il n'**existe** pas : c'est une requête, pas une ligne. Un record ne peut donc jamais désigner un Run qui n'est plus là, ni survivre à une règle d'éligibilité qui change.

Le **Config bucket** — les cinq colonnes du milieu — est ce qui rend deux Runs comparables. Un record de 60 s avec ponctuation n'a rien à voir avec un record de 15 mots sans : ils ne sont pas dans le même seau, et le GROUP BY le dit sans qu'aucune règle ait à être écrite ailleurs.

Les six migrations racontent, dans l'ordre, ce que le projet a appris :

MIGRATION	CE QU'ELLE A CHANGÉ, ET POURQUOI
0001	Le schéma initial : la table runs et ses deux index. Le PB est dérivé dès le premier jour.
0002	Le journal de frappe et la provenance du Run sont persistés — la matière première du Replay et de l'analyse.
0003	Le texte cible est persisté verbatim (ADR 0001) : une citation ne se régénère pas depuis une graine, et l'algorithme de génération peut changer. Un Replay doit rejouer le texte d'alors, pas celui d'aujourd'hui.
0004	La progression du cursus, une ligne par joueur. Un exercice de leçon n'entre jamais dans runs.
0005	Les Quotes cessent d'être éligibles au record (ADR 0003) : leur longueur varie d'un Run à l'autre sans que le bucket le capture, ce qui rendait les comparaisons fausses. Les Runs déjà en base sont corrigés.

MIGRATION CE QU'ELLE A CHANGÉ, ET POURQUOI

- 0006 Le solo perd son décompte de 3 s (ADR 0004) : $t=0$ passe à la première frappe. Ce n'est pas un changement d'interface mais **d'unité de mesure** — les vitesses d'avant ne sont plus comparables, donc tous les records existants sont remis à zéro.

Tableau 3. – Les six migrations. Les deux dernières ne touchent aucune colonne : elles corrigent des données devenues fausses. C'est le genre de correction qu'une table de records dédiée aurait rendue bien plus pénible.

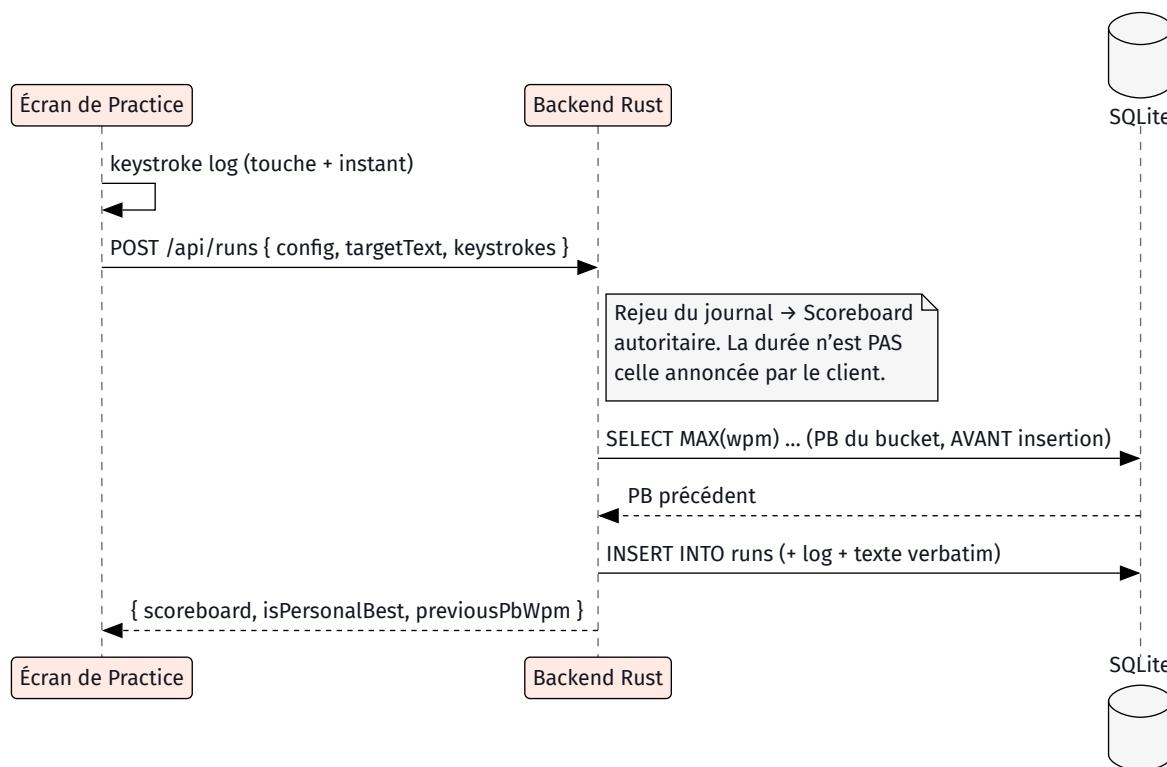


Figure 42. – La soumission d'un Run solo. L'ordre compte : le PB précédent est lu **avant** l'insertion, sinon le Run qu'on vient d'écrire serait son propre record à battre et aucun record ne serait jamais annoncé.

6.6 Sécurité et frontière de confiance

Le récit de la section 5 raconte comment cette frontière a été découverte. Voici son état final, en cinq lignes.

L'identité — un jeton d'accès Discord valide ne suffit pas — le serveur redemande à Discord **quelle application** l'a émis, et refuse ceux qui ne viennent pas de la sienne. Sans cette vérification, la frontière ne serait pas « un joueur de cette Activity » mais « un utilisateur de Discord ».

Les plafonds — 120 requêtes API par joueur et par minute, 30 pour le proxy de citations parce qu'il consomme un quota mensuel partagé, 20 connexions WebSocket. Un dépassement répond 429 explicitement — il ne coupe pas la connexion en silence.

Les en-têtes — politique de sécurité de contenu, nosniff, no-referrer, posés **après** le service de statiques pour couvrir aussi le document HTML — le seul endroit où une politique de contenu sert vraiment. Avec une exception nommée et permanente : frame-ancestors autorise Discord, et doit continuer à le faire. Une politique qui interdit l'encadrement rendrait le jeu injouable.

Les entrées réseau — identité assainie, mot de Spam validé, réglages contraints à des listes de valeurs fermées plutôt qu'à des bornes. Une valeur hors liste est ignorée, pas ramenée dans les clous.

Le résultat d'une course — recomputé, toujours. Une fin annoncée est **rejouée** contre le texte du serveur avant d'être crue, et une fin non confirmée devient un abandon — le seul verdict à la fois vrai et débloquent pour les autres.

EN CLAIR Pourquoi une politique de contenu, alors que Discord en applique déjà une

Dans l'iframe, Discord impose la sienne, et elle est plus serrée que tout ce que le projet pourrait écrire. Mais l'application est aussi joignable **directement**, par l'adresse du tunnel, dans un navigateur ordinaire — et là, plus rien ne s'applique. La politique du serveur couvre cette seconde porte. C'est la même logique que partout ailleurs dans le projet : ce n'est pas parce qu'un chemin est protégé que l'autre l'est.

6.7 Tests et assurance qualité

382 tests côté TypeScript, 159 côté Rust. Le chiffre intéressant n'est pas leur somme mais leur **répartition** : l'essentiel porte sur les deux domaines purs, c'est-à-dire sur le code qui décide. C'est ce que la frontière `core/` ↔ `ui/` achète — un domaine sans DOM se teste sans navigateur, et un domaine sans socket se teste sans réseau.

Trois filets se superposent, et chacun attrape ce que les autres laissent passer.

Les tests unitaires — le comportement de chaque module, dans son langage.

Les vecteurs partagés — trois fichiers de données à la racine, lus par `vitest` et par `cargo test`. Ils couvrent le calcul du Scoreboard, la détection d'échec et les Réglages de salon. C'est le seul filet possible pour un miroir manuel : une divergence entre les deux langages devient un test rouge d'un côté, au lieu d'un comportement silencieusement faux.

La chaîne d'intégration — audit des dépendances des deux côtés, lint, tests, build. Le travail de publication en dépend, ce qui rend mécaniquement impossible de publier avec un test rouge.

Ce que la chaîne ne garantit pas, et qu'il faut dire aussi. Elle ne lance pas le jeu : rien n'y clique, rien n'y tape. Elle ne teste rien **dans** Discord — ni la politique de contenu de l'iframe, ni la Rich Presence, ni le handshake d'identité réel, qui exigent tous un client Discord et un tunnel. Et elle ne compile pas ce PDF. Ces vérifications-là sont manuelles, et le sont assumément : les automatiser demanderait un client Discord sur un runner, ce qui n'existe pas.

7 Limites connues et suites

Ce que le projet ne fait pas, ou fait avec une réserve. Aucune de ces limites n'est une surprise : chacune est une conséquence assumée d'un choix décrit plus haut.

Les en-têtes de sécurité attendent leur validation — le code est écrit et servi, mais les critères d'acceptation de l'issue 152 — une course complète dans Discord, avatars affichés, aucune violation en console — demandent une session de test manuelle dans le client Discord. C'est la seule issue encore ouverte du dépôt.

La publication automatisée n'a jamais publié — le travail `release` est écrit, déclaré, et dépend bien des deux autres — mais aucune exécution du dépôt ne l'a jamais fait tourner. Les trois versions publiées (`v0.0.1` à `v0.0.3`) sont antérieures à l'issue 117 qui l'a ajouté, et ne portent donc aucune archive. Le mécanisme est en place et sa garde fonctionne ; ce qui manque est la première version qui la franchisse. C'est aussi pour cette raison que les deux captures de cette section montrent deux travaux et une version sans archive, et non ce que la mécanique produira.

Le tunnel change d'adresse — un tunnel rapide `cloudflared` ne garde pas son URL, et l'URL Mapping du portail Discord doit être remis à jour à la main à chaque session de test. Un tunnel nommé, ou un vrai nom de domaine, réglerait ça — c'est un choix d'hébergement, pas de code.

Il n'y a pas de leaderboard — l'ADR 0012 dit à quelles conditions il pourrait exister — le `recompute` autoritaire suffit à lui faire confiance, sans anti-triche dédié. Ce qui manque est le produit, pas la sécurité.

Les Rooms meurent avec le processus — elles vivent en mémoire, et un redémarrage du serveur vide tous les salons. Les `Runs solo`, eux, sont en base et survivent. C'est ce même choix qui explique pourquoi `RaceOver` porte tous les résultats d'un coup (section 6.3) — il n'y a rien à redemander après coup.

Le miroir reste manuel — les vecteurs partagés attrapent les divergences de **comportement** sur ce qu'ils couvrent, mais pas un champ ajouté d'un seul côté. La couverture des vecteurs est le vrai plafond.

La dette délibérée est écrite — sept raccourcis portent dans le code un commentaire nommant leur plafond et leur chemin de sortie, rassemblés dans `PONYTAIL-DEBT.md`. Deux d'entre eux ont la même cause — la vitesse en direct des adversaires et le duel de `Play of the Game` sont calculés sur l'horloge locale de chaque client, avec environ 2 % de dérive, assumé pour des chiffres d'ambiance et à réaligner sur `RaceStart` si l'exactitude devient un enjeu.

Un mot sur ce que « limite connue » veut dire ici. Une limite écrite, avec sa cause et son chemin de sortie, coûte une ligne à quiconque la rencontre. La même limite non écrite coûte une session de débogage — et le projet en a fait l'expérience assez de fois pour préférer la ligne.

8 Annexes

8.1 Les dix-neuf décisions d'architecture

Chacune répond à une question qui ne se lit pas depuis le code seul. Le texte complet, avec son contexte et ses conséquences, vit dans Docs/adr/.

ADR	DÉCISION
0001	Le texte cible est persisté verbatim, pas régénéré depuis sa graine.
0002	L'identité d'affichage n'est jamais persistée.
0003	Les Quotes ne produisent jamais de record.
0004	Le solo démarre sans décompte : $t=0$ est la première frappe.
0005	Le Trigram Drill est un Mode distinct, pas une extension de Drill.
0006	Le cursus Apprendre passe à 100 leçons — une extension, pas une v2.
0007	Le décompte de Race est un réglage produit, pas une unité de mesure.
0008	Une Room est identifiée par une clé : salon vocal ou Code de partie.
0009	Une Race n'a pas de Mode — elle a une Source de texte.
0010	RaceOver porte les résultats complets, pas seulement l'ordre.
0011	Play of the Game : un duel choisi par le serveur, rejoué sur une horloge commune.
0012	Un leaderboard ferait confiance au recompute autoritaire, sans anti-triche dédié.
0013	Difficulté Normal / Expert / Master, et l'état Échec qui en découle.
0014	En plein écran, le contenu se met à l'échelle — il ne défile pas.
0015	Floor is lava : un Mode de jeu, un axe à part, sans ligne d'arrivée.
0016	Spam : un Mode de jeu, texte infini, deux façons de gagner.
0017	Le Mode de jeu est une table déclarée ; le Réglage de salon a un contrat de retour explicite.
0018	Les journaux sous Mode de jeu sont tronqués, et la durée imposée.
0019	Le Code de partie est masqué par défaut, sur l'écran de chacun.

Tableau 4. – Les dix-neuf ADR. Elles ne sont pas rangées par thème mais par ordre d'apparition : la numérotation est chronologique, et c'est délibéré — une ADR dit ce qu'on savait au moment où on a tranché.

8.2 Les documents légaux

Trois fichiers de racine, exigés par le portail développeur Discord pour qu'une Activity puisse être publiée. Ils ne sont pas décoratifs : sans eux, la fiche d'application reste incomplète.

FICHIER	RÔLE
LICENSE	La licence sous laquelle le code est publié.
TERMS.md	Les conditions d'utilisation, liées depuis la fiche d'application.
PRIVACY.md	La politique de confidentialité. Elle a un contenu réel : le projet stocke des identifiants Discord et des journaux de frappe, et n'a jamais persisté d'identité d'affichage (ADR 0002).

Tableau 5. – Les trois documents exigés par le portail.

8.3 Où aller ensuite

DOCUMENT	CE QU'IL CONTIENT, ET QUE CE PDF N'A PAS
CONTEXT.md	Le glossaire de domaine complet — une quarantaine de termes, chacun avec les mots interdits à sa place. Ce document en a condensé douze.
Docs/API.md	Le contrat HTTP détaillé : les corps de requête et de réponse, champ par champ. Ce document n'a listé que les routes.
Docs/adr/	Les dix-neuf décisions in extenso, avec leur contexte et leurs conséquences.
Docs/PHASE2.md	Le plan de la phase multijoueur, tel qu'il a été posé avant d'être construit.
README.md	La mise en place locale, et le runbook complet du portail Discord : tunnel, URL Mappings, App Testers, pièges.
CHANGELOG.md	Les notes de version. C'est ce fichier, et non les titres de commits, qui alimente les notes de publication.

Tableau 6. – Les six documents qui prolongent celui-ci. Aucun de leurs blocs n'a été recopié ici : ce document condense ou renvoie, jamais les deux — deux copies d'un même paragraphe divergent à la première retouche.